Modern Frontend Development

Welcome to the Jungle



ABOUT ME

Jonas Bandi
jonas.bandi@ivorycode.com
Twitter: @jbandi



- Freelancer, in den letzten 10 Jahren vor allem in Projekten im Spannungsfeld zwischen modernen Webentwicklung und traditionellen Geschäftsanwendungen.
- Dozent an der Berner Fachhochschule seit 2007
- In-House Kurse & Beratungen zu Web-Technologien im Enterprise: UBS, Postfinance, Mobiliar, AXA, BIT, SBB, Elca, Adnovum, BSI ...

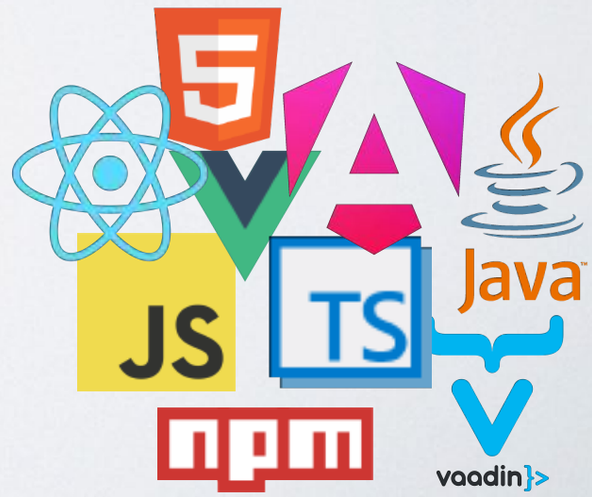
Berner Fachhochschule
Haute école spécialisée bernoise
Bern University of Applied Sciences

letsboot.ch
swiss dev training

digicomp

JavaScript / Angular / React / Vue / Vaadin
Schulung / Beratung / Coaching / Reviews

jonas.bandi@ivorycode.com



Last Year's Agenda

DAY 1

ES2015+ JS

Modern JavaScript

TS

TypeScript

Async JavaScript: Callbacks, Promises, RxJS

Web Platform APIs

DAY 2

npm Vite

Tooling for a Modern Front-End Build

Styling Mechanisms

HTML

Concepts of Single Page Applications

Overview "Big 3" Frontend Frameworks

Intro Web-Components

Development has changed since last year ...





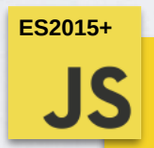
In the brave new agentic world:
syntax does not matter ... frameworks do not matter ...

This Year's Agenda

DAY 1



Tooling for a Modern Front-End Build



Modern JavaScript

Async JavaScript: Callbacks, Promises, RxJS

DAY 2



TypeScript

Web Platform APIs



Concepts of Single Page Applications



Overview "Big 3" Frontend Frameworks



Styling Mechanisms


Vite
webpack
MODULE BUNDLER
Playwright
cypress.io
JS
TS
Jest
JS
npm
CHROME VANADIUM
Tooling for a modern
Front-End Build

JavaScript Ecosystem Today

Worth a look ...

AI as the new gatekeeper for the tooling ecosystem ...

What Claude Code Actually Chooses:
https://amplifying.ai/research/claude-code-picks

What Codex Actually Chooses:
https://amplifying.ai/research/codex-vs-claude-code-picks

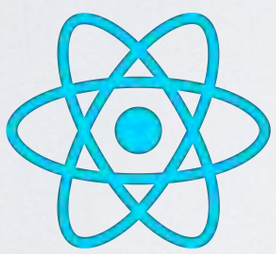
Default Tooling

Setting up a modern JavaScript project is difficult.
Most framework ecosystems provide a scaffolding to make this task easier.



```
npm install -g @angular/cli
ng new awesome-ng
```

Angular CLI: https://angular.io/cli



using vite:

```
npm create vite@latest my-react-project -- --template react-swc-ts
```

The official React documentation is recommending a framework: Next.js or ReactRouter
https://react.dev/learn/creating-a-react-app#full-stack-frameworks



```
npm init vue@latest
```

create-vue: https://github.com/vuejs/create-vue (based on vite)

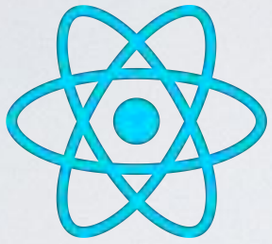
Vite for Build-Tooling



"The Build Tool for The Web"

https://vite.dev/

Vite is the default tooling for most modern frontend framework setups:











qwik



SVELTE KIT



SVELTE



SOLIDSTART



SOLIDJS





RedwoodJS



TanStack Start

https://viteconf.org/

Frontend Build Tools/Bundlers

Legacy:

webpack
MODULE BUNDLER

REQUIRE.JS

browserify

Gulp

GRUNT

"ad-hoc bundling"

Snowpack
(using esbuild) (deprecated)
(using webpack or rollup for production)

Modern:

Vite
(using esbuild)
(using rollup for production)

PARCEL
Blazing fast, zero configuration web application bundler
(using SWC)

Lightning CSS
(written in Rust) https://lightningcss.dev/

Frontier:

Biome
TOOLCHAIN OF THE WEB
https://biomejs.dev/
(written in Rust)

Turbopack
https://turbo.build/pack
(written in Rust)

Bun
(written in Zig)
https://bun.sh/

Rspack
(written in Rust)
https://rspack.rs/

Rsbuild
https://rsbuild.rs/

Low-Level:

rollup.js

esbuild
https://esbuild.github.io/
(written in Go)

SWC
https://swc.rs/
(written in Rust)

Rolldown
rollup written in Rust
https://rolldown.rs/

Oxc

Niche:

jspm.io

WMR (preactjs/wmr)
(using esbuild / using rollup for production)

Bazel

@web/dev-server
https://modern-web.dev/
(using esbuild / using rollup for production)

https://bundlers.tooling.report/
https://css-tricks.com/comparing-the-new-generation-of-build-tools/

Worth mentioning: TypeScript compiler in Rust
https://bun.com/blog/bun-v1.4.3
https://blog.cloudflare.com/typescript-rust-port/

The Future?

`vite+` - The Unified Toolchain for the Web

https://viteplus.dev/






Vite Vitest Oxc Rolldown pnpm

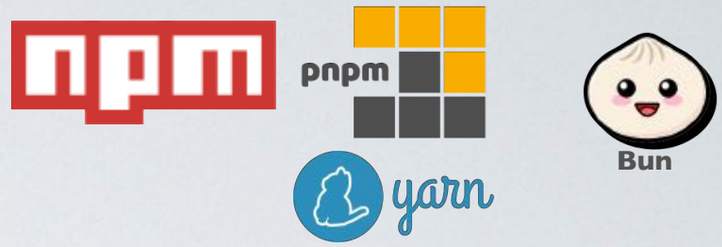
October, 2024: VoidZero founded by Evan You (creator of Vue and Vite).
A company dedicated to building unified development toolchain for the JavaScript ecosystem.

September, 2026: Vite+ 1.0 released: the open-source unified toolchain for the web
https://voidzero.dev/posts/announcing-vite-plus-1-0

https://voidzero.dev/

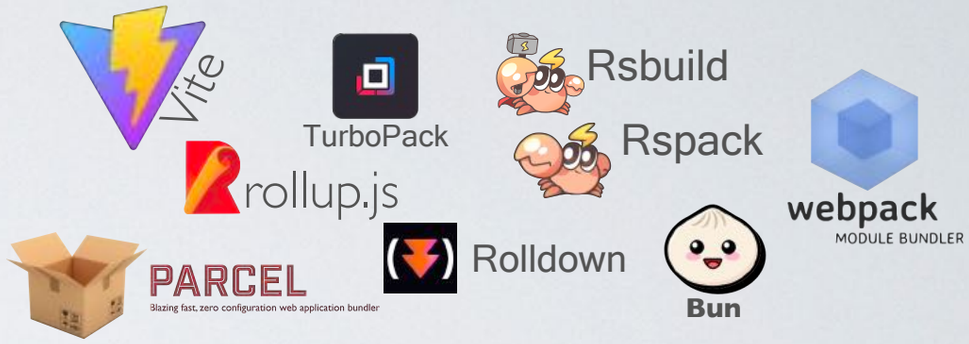
Dependency Management & Script Runner

Declare & Resolve project dependencies.
Orchestrate other tools.



Build Automation & Bundling

Build one or several bundled asset files for deployment.
Resolve module dependencies.
Optimize asset files for production.



Transpilation

Transform development sources (ES2015+ / TS / JSX) into ES5.



Static Type-System

Check type correctness of source code with (optional) static types at development time.



Linting

Static code analysis.



Source Formatting

Automtic formatting of source code.



Active Ecosystem



www.npmjs.org

By the numbers

Packages

3,498,976

Downloads · Last Week

69,391,881,717

Downloads · Last Month

313,319,390,432



Node Package Manger

- Packages can be local (for the current project) or global
- `package.json` describes a package or project including it's dependencies
- packages are stored in `node_modules`
- hierarchical dependencies: dependencies can include their own dependencies
  (you can have several versions of a package in your project)
- Dependencies are versioned according to semantic versioning (https://semver.npmjs.com/)
- Starting from npm 5, exact versions are listed in `package-lock.json`
  (note: `npm install` still upgrades top-level packages if no exact version is in `package.json`)
- Public Repository: npmjs.org
- Config: `.npmrc`

Tip: `npm config set save-exact true`

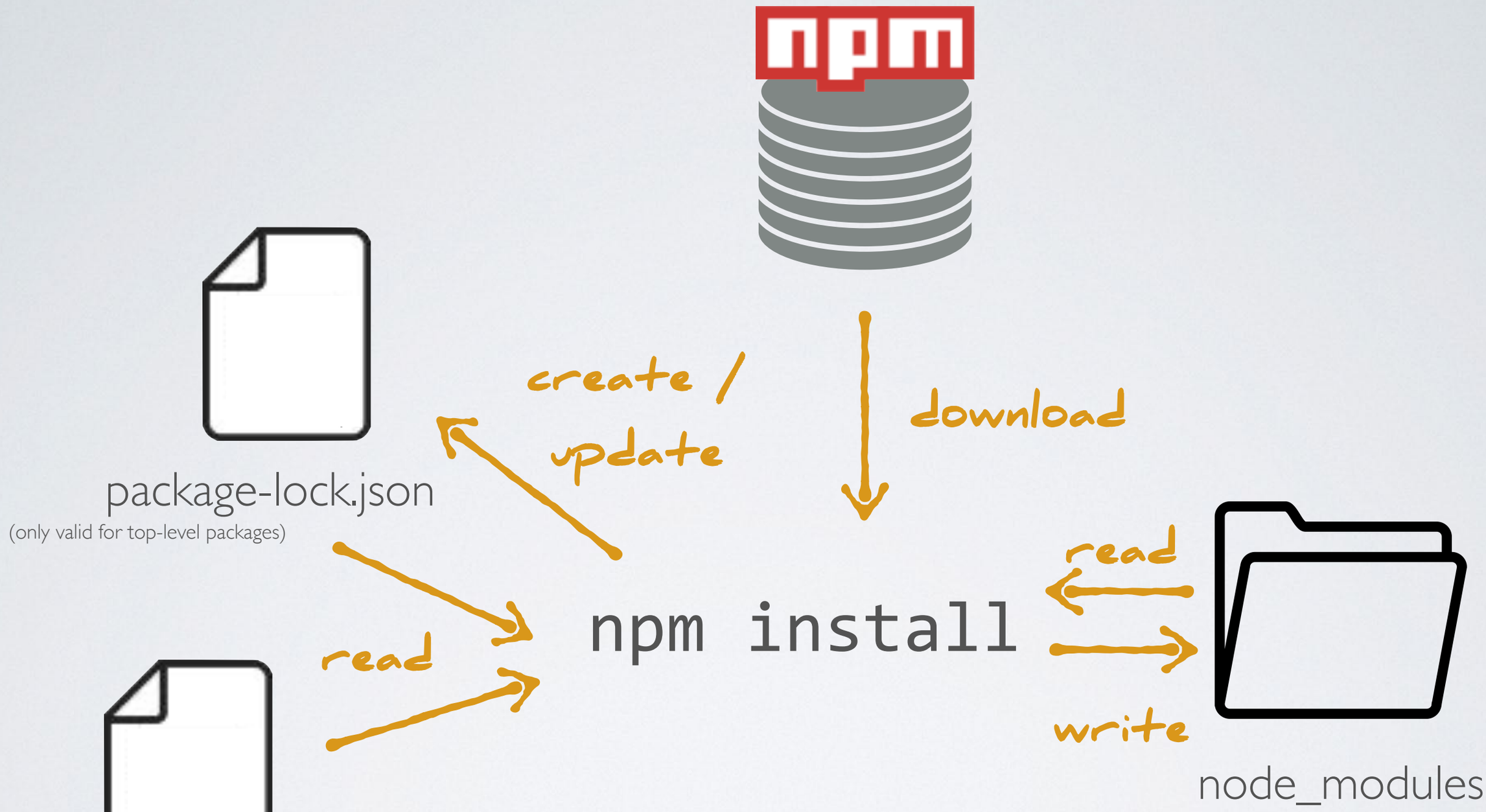
See: https://semver.npmjs.com/

Typical commands:

```
npm search
npm info
npm install
npm uninstall
npm list
npm update
npm init
npm root
npm config
npm ci
npm audit
```

Flags:

```
--global / -g
--help
--save-dev / -D
--save-exact / -E
```



⚠️ Be aware:

`npm install` does update your `package-lock.json` to keep it "automatically" in sync with the `package.json` 🤯

To enforce a deterministic dependency resolution use `npm ci` instead.

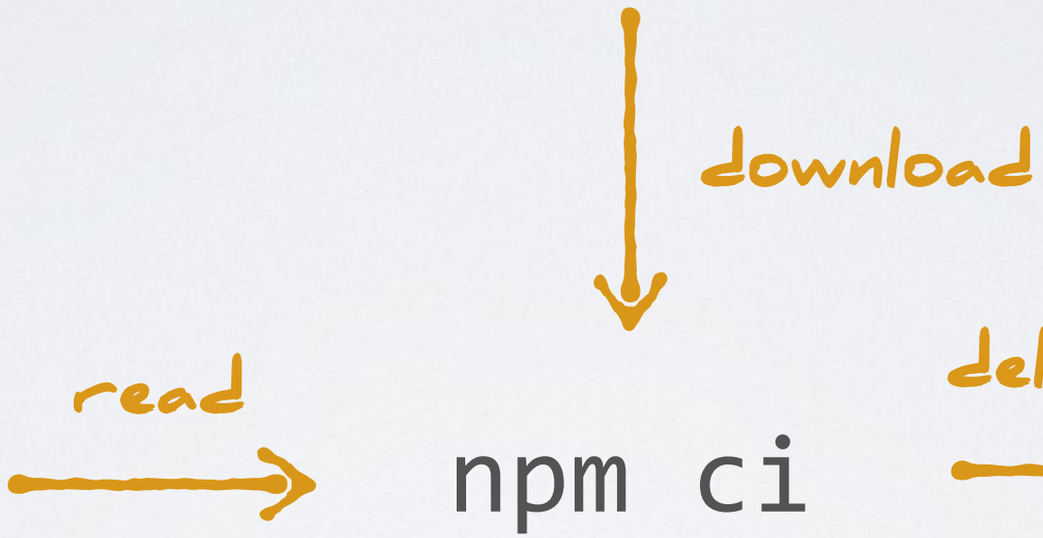
Clean Install:
Projects should prefer
`npm ci` to set up a
deterministic
dependency tree.



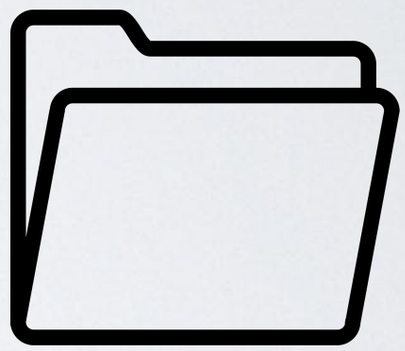
package-lock.json

(only valid for top-level packages)





(error if `package-lock.json` and
`package.json` do not match)



node_modules

Corresponding commands with other package managers:
`pnpm ci`

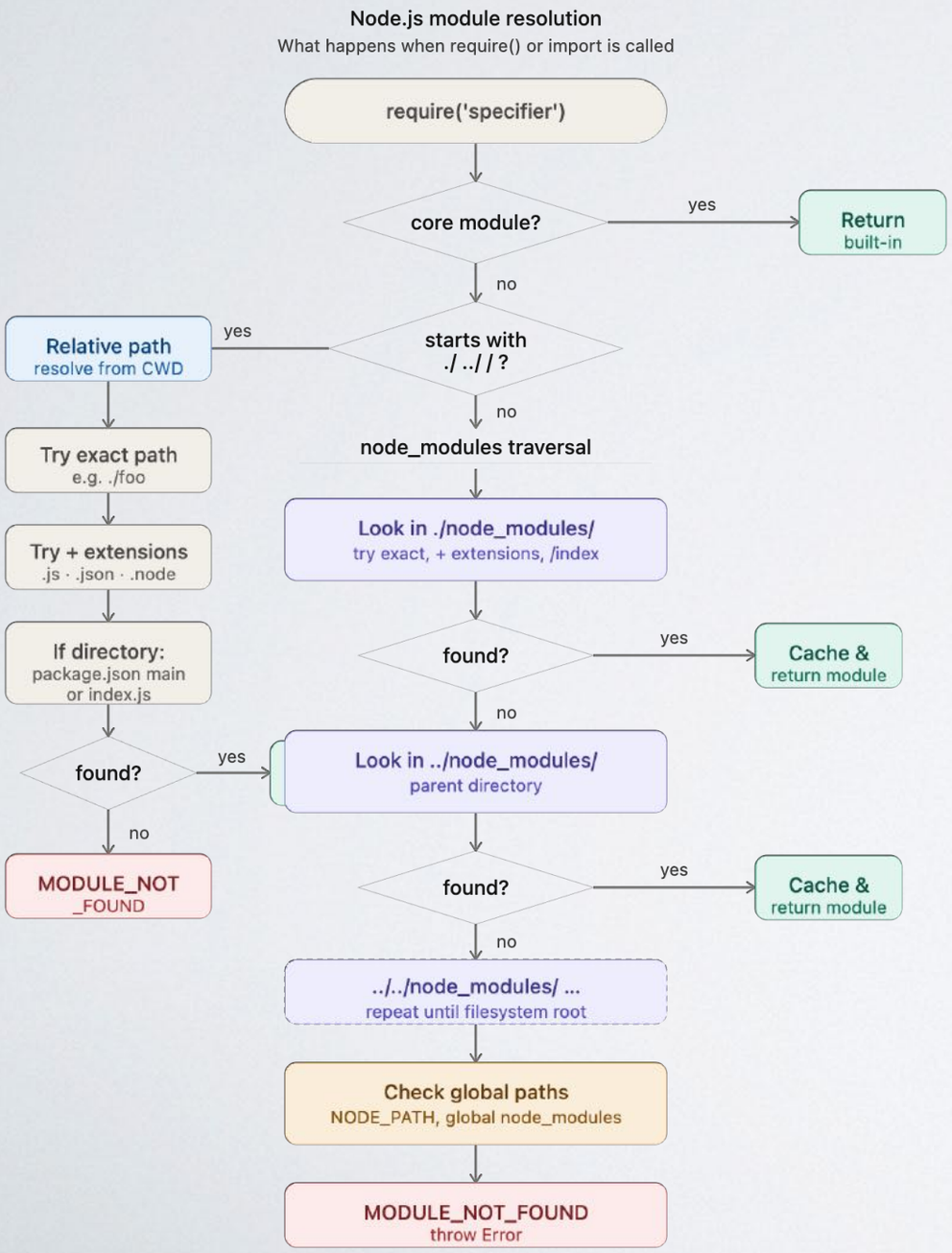
https://docs.npmjs.com/cli/ci

Node Module Resolution

https://nodejs.org/api/modules.html#modules_all_together

(this mechanism is also used by bundlers like Webpack together with ES-modules)

What happens behind: `import x from moduleX` ?




```
my-app/
├── package.json          ← "type": "module"
├── index.js              ← import moduleA from 'moduleA'
│                           import moduleB from 'moduleB'
│                           import _ from 'lodash'
├── node_modules/
│   ├── moduleA/
│   │   ├── package.json
│   │   ├── index.js      ← import _ from 'lodash' → finds v4 locally
│   │   └── node_modules/
│   │       └── lodash@4.17/
│   ├── moduleB/
│   │   ├── package.json
│   │   ├── index.js      ← import _ from 'lodash' → finds v3 locally
│   │   └── node_modules/
│   │       └── lodash@3.10/
│   └── lodash@4.17/      ← shared, used by index.js directly
```

The Node module resolution enables the coexistence of different versions of a packages in an application via nesting.

Package managers typically dedupe the tree.

- npm flattens the tree (hoisting).
- pnpm uses symlinks that point to the actual package files in a hidden directory where each package/version exists exactly once.

npm Ecosystem: Challenges

Sheer amount of libraries/frameworks/tools:
- how to choose?

Short life-spans of libraries/frameworks/tools:
- how to avoid avoid risk for switching/re-writing
- how to avoid re-learning for every project?

Controlling and auditing dependencies:
- How to avoid security risks ("supply chain attacks")?
  - 2016: Leftpad Incident
  - 2018: Event-Stream Hacked
  - 2021: Malware found in coa (6 mio weekly downloads) and rc (14 mio weekly downloads)
  - 2025: Several organized supply chain attacks on a wide range of packages (shai-hulu, s1ngularity -180+ npm packages with over 2.5 billion weekly downloads compromised)

Good read: https://hackernoon.com/im-harvesting-credit-card-numbers-and-passwords-from-your-site-here-s-how-9a8cb347c5b5

The JavaScript Dependency "Situation"

```
>npx create-react-app react-project

...

added 1909 packages from 732 contributors
found 0 vulnerabilities

> du -hs react-project/node_modules/
252M    react-project/node_modules/
```

```
>ng new angular-project

...

added 1600 packages from 1278 contributors
found 0 vulnerabilities

> du -hs angular-project/node_modules/
523M    angular/node_modules/
```

```
> vue create vue-project

...

added 1324 packages from 987 contributors
found 0 vulnerabilities

> du -hs vue-project/node_modules/
175M    vue-project/node_modules/
```

Note: recent project-starters like **`create-vite`** have massively reduced the initial dependencies ...

... recent improvements:

```
npm create vite@latest my-react-app -- --template react-ts

npm ci

added 88 packages, and audited 89 packages in 1s
```

```
npx @angular/cli@latest new awesome-ng
...
npm ci
...
added 920 packages, and audited 921 packages

du -hs node_modules/
375M    node_modules/
```

```
npm init vue@latest awesome-vue

Vue.js - The Progressive JavaScript Framework

✔ Add TypeScript? … Yes
✔ Add JSX Support? … No
✔ Add Vue Router for Single Page Application development? … Yes
✔ Add Pinia for state management? … No
✔ Add Vitest for Unit Testing? … Yes
✔ Add Cypress for End-to-End testing? … No
✔ Add ESLint for code quality? … Yes
✔ Add Prettier for code formatting? … No

...

added 331 packages, and audited 332 packages in 56s
```

Maintenance

A "small" Angular project from 2018:

```
>cd angular-project-from-2018
>nvm use 8
>npm install

...

added 1288 packages from 1314 contributors
found 1396 vulnerabilities
(985 low, 18 moderate, 391 high, 2 critical)
```

However `npm audit` vulnerabilities might not be a good metric: https://overreacted.io/npm-audit-broken-by-design/

Real-World example:
In-house React component library, 2 years untouched:

```
>npm audit

...

found 165426 vulnerabilities (109880 low, 526 moderate, 55018 high, 2 critical) in 4149 scanned packages
  run `npm audit fix` to fix 164670 of them.
  678 vulnerabilities require semver-major dependency updates.
  78 vulnerabilities require manual review. See the full report for details.
```

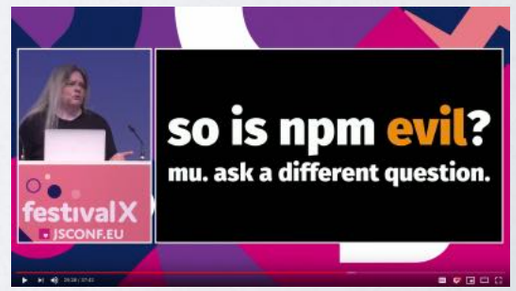
However npm audit vulnerabilities might not be a good metric: https://overreacted.io/npm-audit-broken-by-design/

npm belongs to a private company!

In April 2020 npm was acquired by GitHub.
(GitHub was acquired by Microsoft in 2018)

- the npm registry is a *centralized* system owned and operated by a private company (GitHub/Microsoft)

- the npm registry is not open-source



JSConf EU 2019: the economics of open source
https://www.youtube.com/watch?v=MO8hZIgK5zc

Alternatives to npm

pnpn: https://pnpm.js.org/
A drop-in-replacement which keeps node_modules in a central local repository (similar to maven).

Bun

Bun is also a package manager
https://bun.com/package-manager

yarn: https://yarnpkg.com
Initially faster and deterministic compared to npm. Today no big difference any more. Also using the public npm registry.

Deno

the future of deno is uncertain!

Deno 2 is also a package manager.
https://deno.com/blog/your-new-js-package-manager

Deno originally promoted to get rid of traditional node_modules / package-managers and to declare dependencies directly via urls..



pnpm

Personal opinion: pnpm should be the default for new projects

https://pnpm.io/

Advantages:

- security
- speed
- disk-space

Disadvantage:
another tool to install and maintain

Commands:

```
pnpm install
pnpm ci
pnpm add
pnpm dlx / pnpx / pnx
pnpm exec
```

https://pnpm.io/pnpm-cli

lockfile: `pnpm-lock.yaml`

config: `pnpm-workspace.yaml`

https://pnpm.io/feature-comparison

Note: npm v12 improved security defaults:
no automatic execution of install scripts, no dependencies to remote or git urls
https://github.blog/changelog/2026-06-09-upcoming-breaking-changes-for-npm-v12/

Enterprise Concerns

There are options for a private npm registry:

- Nexus
  https://help.sonatype.com/repomanager3/nexus-repository-administration/formats/npm-registry
- Artifactory
  https://jfrog.com/help/r/jfrog-artifactory-documentation/npm-registry
- Azure Artifacts:
  https://docs.microsoft.com/en-us/azure/devops/artifacts/overview
- verdaccio: http://www.verdaccio.org/
- Github Packages: https://github.com/features/packages
- Gemfury: https://gemfury.com/

```
npm config list
npm config set registry <registry url>
```

Config files:
`<project>/.npmrc`
`$HOME/.npmrc`

Updating Dependencies

Get information about outdated npm-packages:

```
npm outdated
npm update
```

```
pnpm outdated
pnpm update
```

`npm outdated` also show the latest versions available.

`npm update` only update within the version range specified in `package.json` (use pnpm `update --latest` instead)

Be careful with manually changing `package.json`: remember that `package-lock.json` resp. `pnpm-lock.yaml` has to be updated too ...

There are also 3rd party tools that can help with the task:

```
npx npm-check-updates
```

https://github.com/raineorshine/npm-check-updates

```
npx npm-check -u
```

https://github.com/dylang/npm-check

npm run <script>

execute npm scripts

```
npm start
npm test
npm run build
npm run lint
```

package.json

```
"scripts": {
    "start": "lite-server",
     "lint": "eslint src/**/*.js"
  },
```

In addition to the shell's pre-existing **`PATH`**, **`npm run`** adds **`node_modules/.bin`** to the **`PATH`** provided to scripts.

Passing arguments to the script command:

```
npm run test -- --grep="pattern"
```

https://docs.npmjs.com/cli/run-script

Running npm Binaries

`npx <command>`

(npm 5.2 or later)
https://docs.npmjs.com/cli/v8/commands/npx

- Example: `npx npm-check@latest -u`
- executes npm package binaries
- executes the binaries either from a local node_modules/.bin or from a central cache, installing any packages needed in order for <command> to run.

`npm create <initializer>`

(npm 6 or later)
https://docs.npmjs.com/cli/v8/commands/npm-init

- alias for `npm init <initializer>`
- Example: `npm create vite@latest` => `create-vite` is an npm package
- executes npm package binaries of the package `create-<initializer>`
- executes the binaries either from a local node_modules/.bin, or from a central cache, installing any packages needed in order for <command> to run.

For `npx` and `npm create` it is a good practice to specify the `@latest` package version.

`npm audit` (npm 6 or later)

npm maintains a databse of known JavaScript package vulnerabilities.

Scan your project for vulnerabilities:

```
npm audit
```

Automatically install any compatible updates to vulnerable dependencies:

```
npm audit fix
```

For typical SPA toolchains npm audit reports many "false positives":
- *npm audit: Broken by Design:* https://overreacted.io/npm-audit-broken-by-design/

https://docs.npmjs.com/cli/audit

Managing Node Versions

Tools for managing multiple Node installations

- mise: https://mise.jdx.dev/
- vite+: https://viteplus.dev/guide/env
- nvm: https://github.com/nvm-sh/nvm
- nvm-windows: https://github.com/coreybutler/nvm-windows
- fnm: Fast Node Manager - https://github.com/Schniz/fnm
- Volta - https://volta.sh/
- N - https://github.com/tj/n

Linting



ESLint

http://eslint.org/

ESLint is the state of the art linter for JavaScript, TypeScript and JSX.

ESLint is very configurable and extensible to use a "standard" style or to adapt to your style.

There are popular rule sets by goolge, airbnb or StandardJS.

Getting started: `eslint --init`

Configuration: `eslint.config.mjs`

Biome
TOOLCHAIN OF THE WEB

https://biomejs.dev/

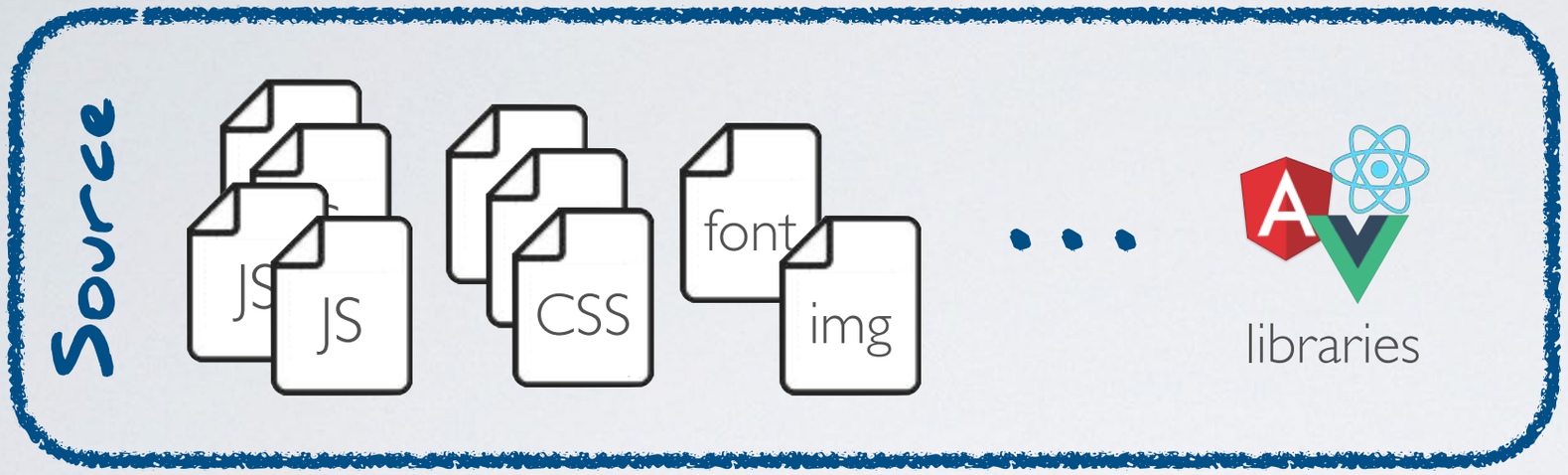
https://oxc.rs/

Biome is an alternative to ESLint, that is quickly gaining popularity ...

Bundling

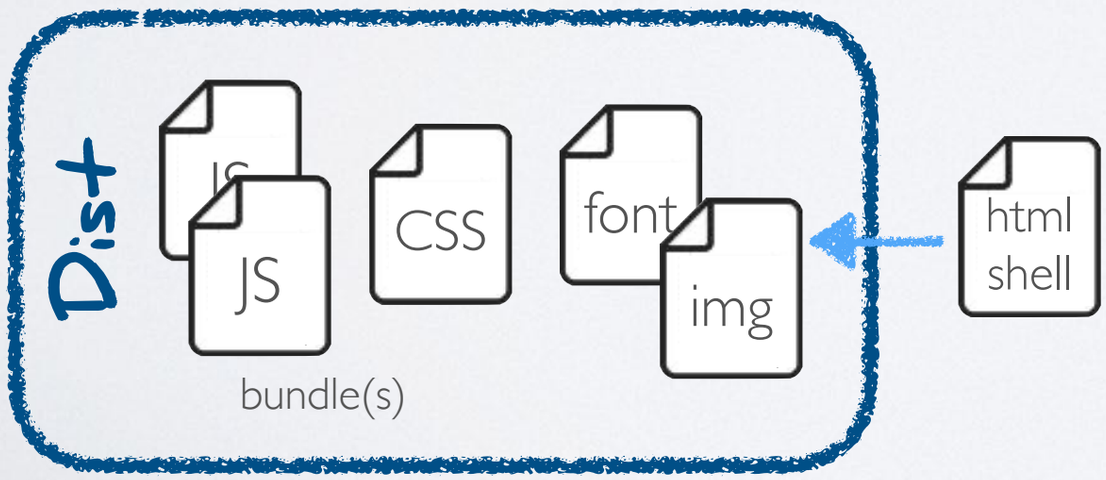


Bundling



Development artifacts:
- fine-grained
- not optimized
- contain unused code





Deployment artifacts:
- coarse-grained
- optimized (size, performance ...)
- unused code is eliminated
- cacheable

Bundling

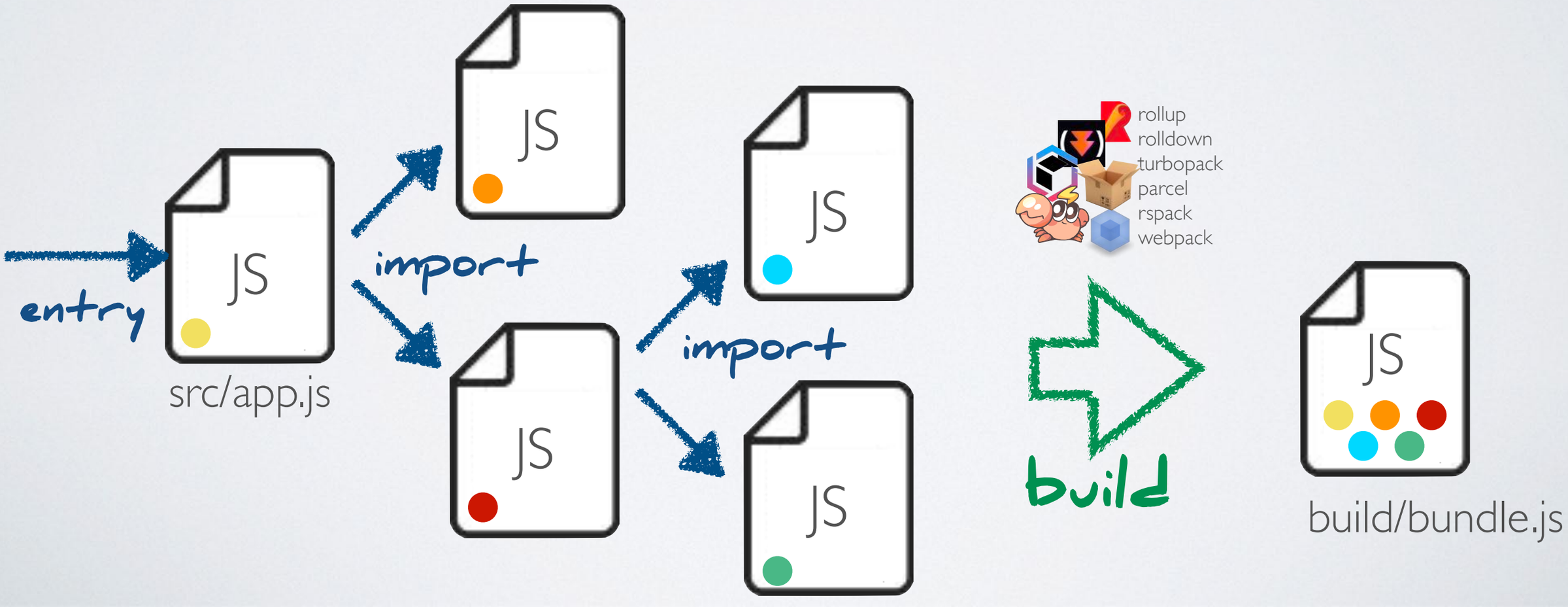
(Development Time Build Toolchain)

- Resources are optimized
  - Code is minimized
    - Bundles are coarse grained, network overhead is minimized
- Cache-Busting mitigates caching problems
- ES2015 modules prevent polluting the global namespace

Modules at Build Time

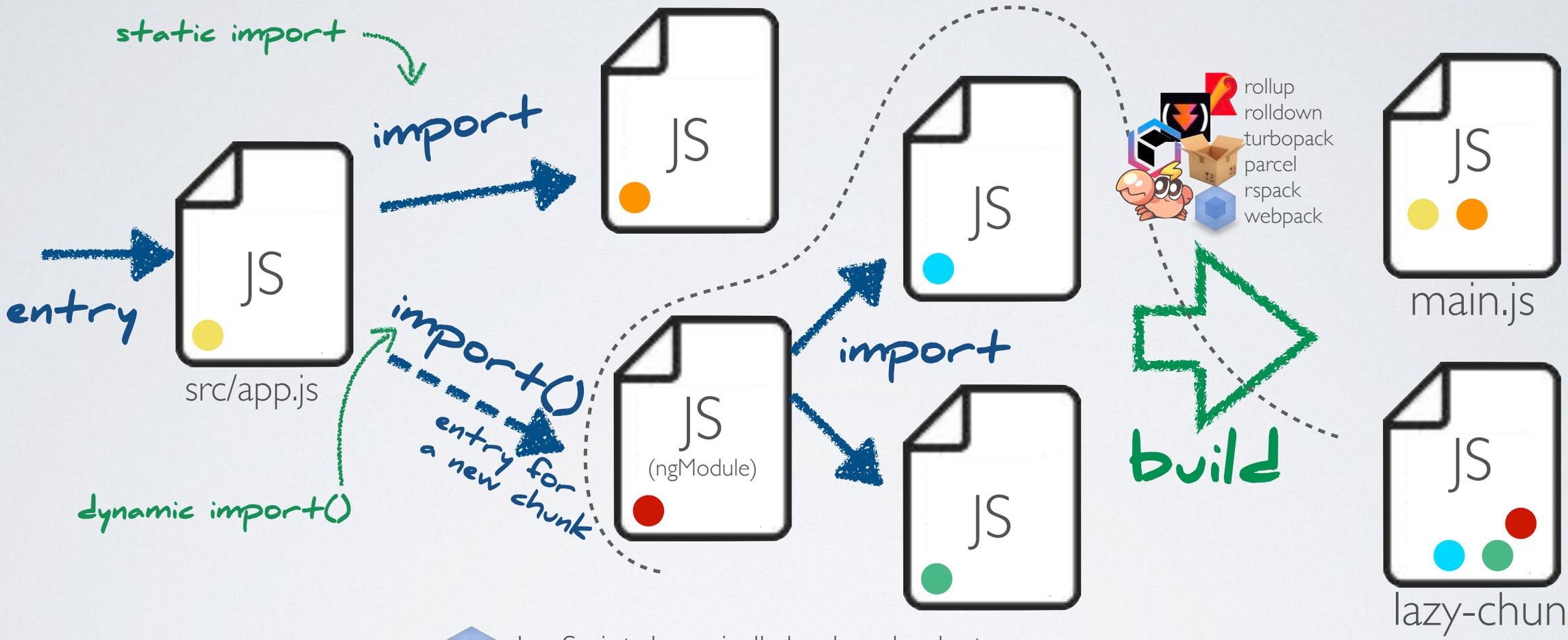
The primary use-case for modules today is build-tooling.
Modern *bundlers* (WebPack, Rollup, Parcel ...) work with modules:

- build a dependency tree based on fine grained ecma script modules (import/export)
- create coarse grained JavaScript bundles which are optimized to be loaded by a browser.



traditional

Lazy Loading



JavaScript dynamically loads a chunk at *runtime*. The code is (partially) generated by the bundler.
Frameworks have additional abstractions (Angular Router, React.lazy ...)

All the application parts (source, npm-packages ...) must be available at build time.

One build creates a *single* deployment artifact consisting of several chunks.



Vite

https://vitejs.dev/

Next Generation Frontend Tooling

A "no-bundler" toolchain: built on top of native ES Modules.
Native ES Modules are loaded during development.
Dependencies are "pre-bundled" with esbuild.
Bundling and optimzation during production build (based on Rollup).

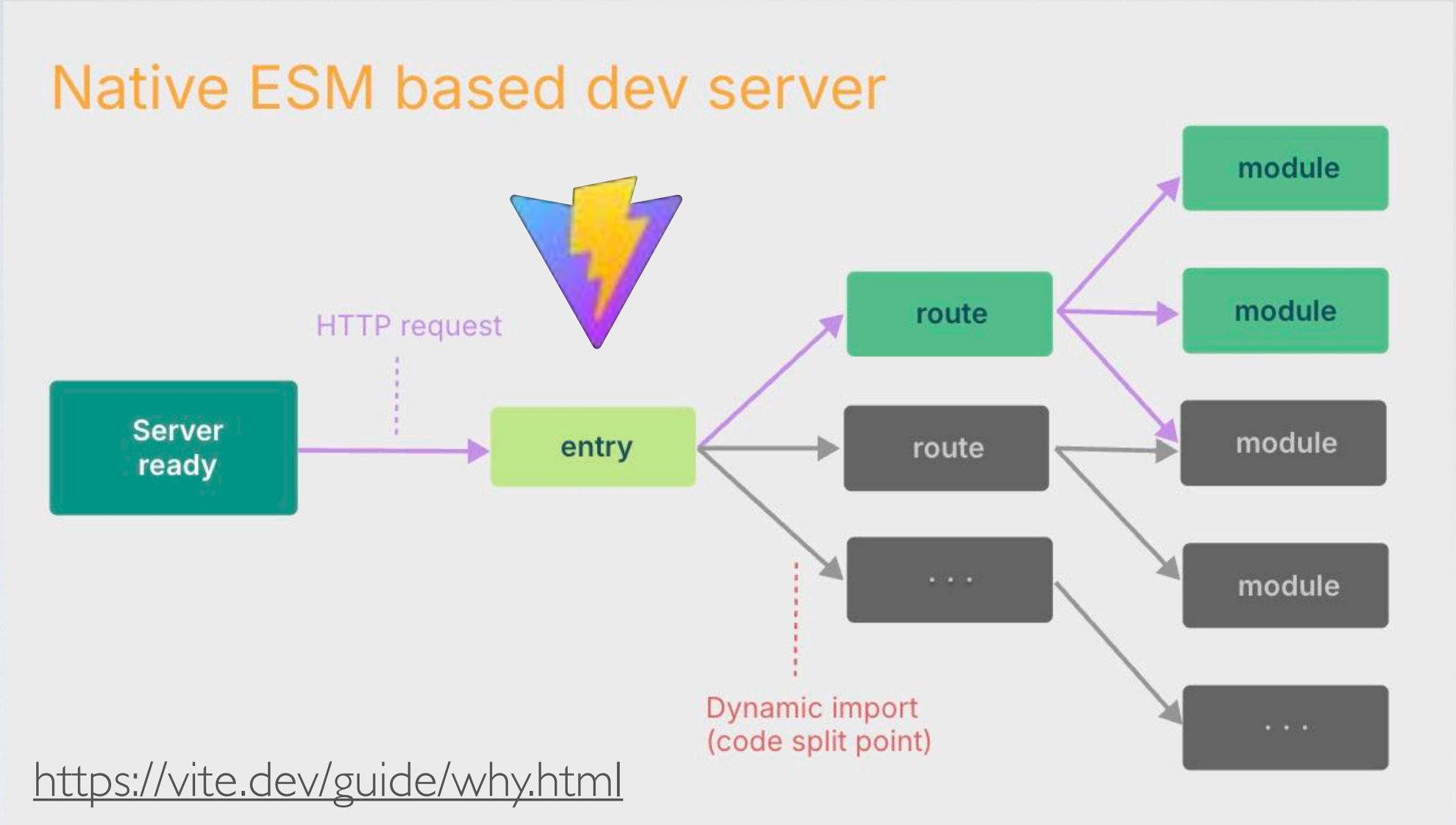
Works with Typescript, React, Preact, Vue, Svelte, lit …

Getting started:

```
npm create vite@latest
```

Vite in 100 Seconds: https://www.youtube.com/watch?v=KCrXgy8qtjM
Intro to Vite: https://www.youtube.com/watch?v=DkGV5F4XnfQ

The unbundled development workflow



https://vite.dev/guide/why.html

The browser takes over the job of the bundler by loading code as esm modules. Vite just transforms single resources on demand.

Vite v7 and before used esbuild for development and rollup for the production build.
Vite v8 uses Rolldown for dev and prod builds.

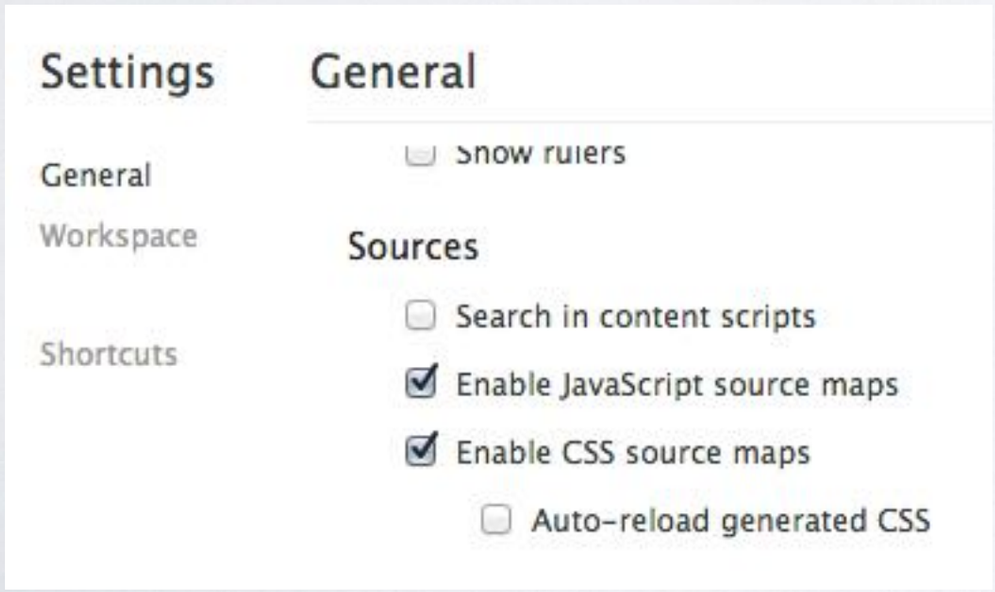
```
vp create vite
// edit vite.config.ts
// experimental: {bundledDev: true}
```

Vite 8.1 released
Experimental Bundled Dev Mode
https://vite.dev/blog/announcing-vite8-1#experimental-bundled-dev-mode

Sourcemaps

- Build: Tools that transform sources generate a map of from the resulting artefact to the original sources:
  - `jquery.js -> jquery.min.js & jquery.min.map`
  - `main.ts -> main.js & main.min.map`

- Runtime: Browsers perform the mapping when debugging
  => the executed code is mapped to the original code, which is displayed for debugging





Testing



JS Testing Scenarios

(where to run tests)

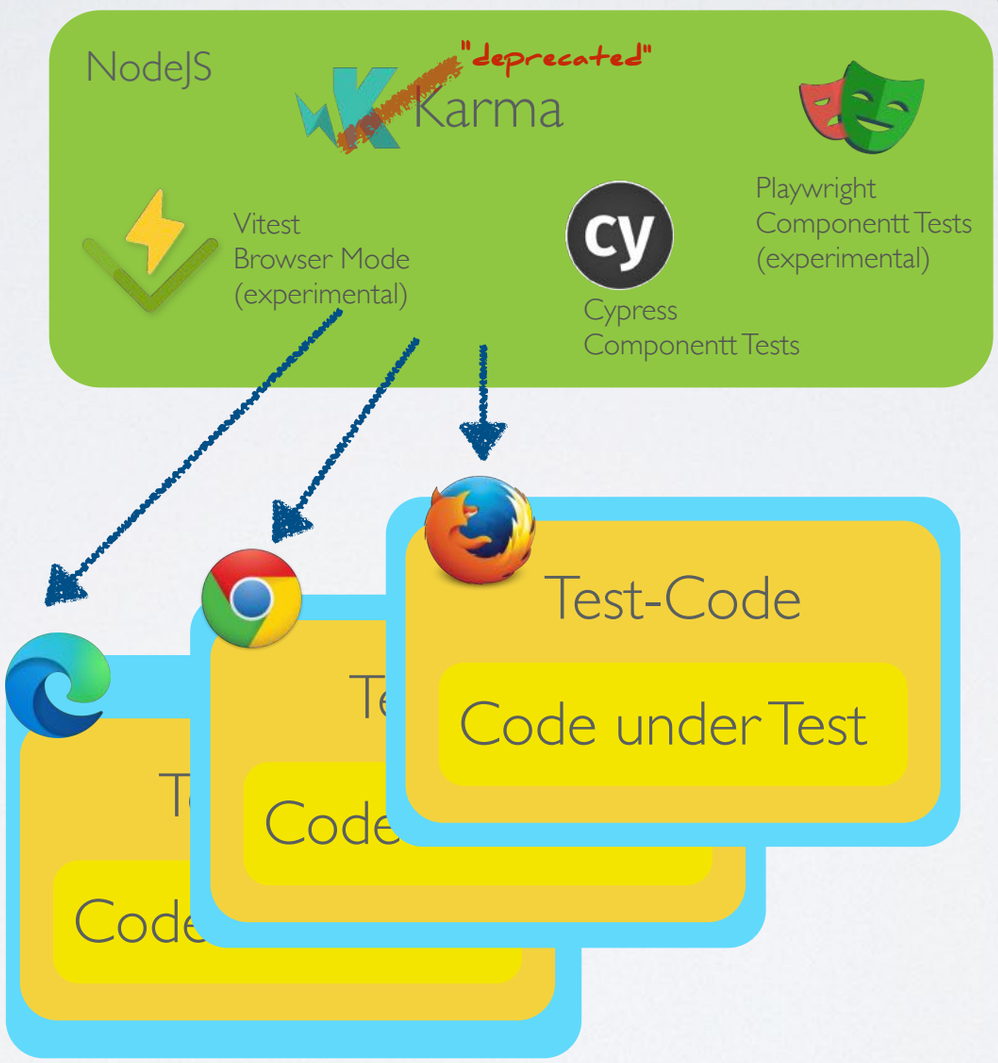
Unit-Tests in Node



Bun and Deno have built in test runners.

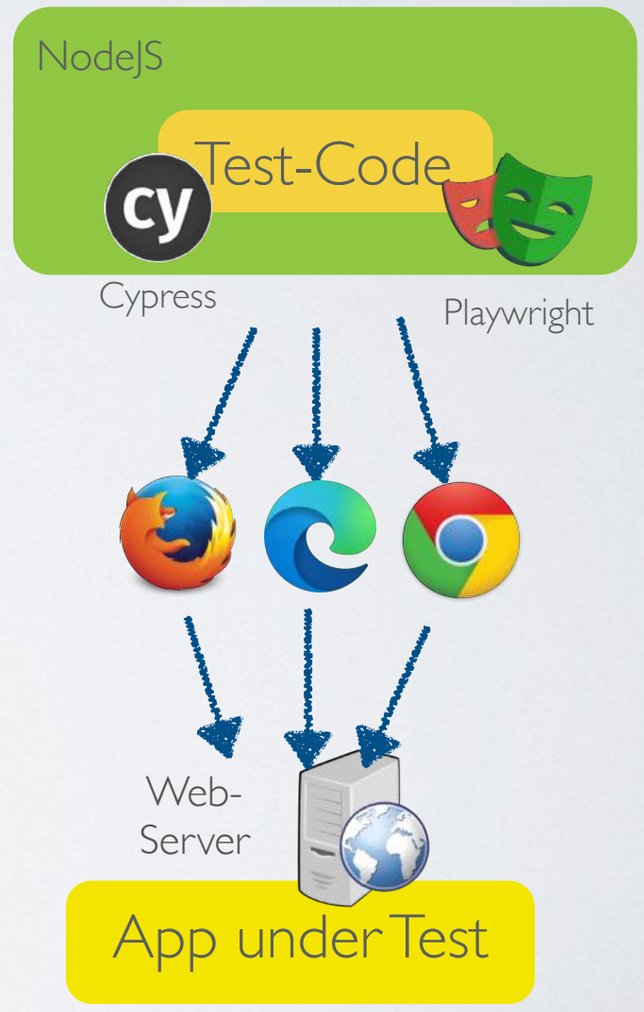
Run tests in node.
Optionaly use happy-dom or jsdom to simulate the DOM.

Unit-Tests in Browser



Run tests in browser.
Optionaly can use browsers in headless mode.

End-2-End Tests with Browser Automation



Script a browser to interact with a deployed app.



Vitest

https://vitest.dev/

A Vite-native unit test framework. It's fast!

- "Zero Config": Out of the box support for ESM, TypeScript, JSX
- Re-using Vite configuration
- Jest Compatible

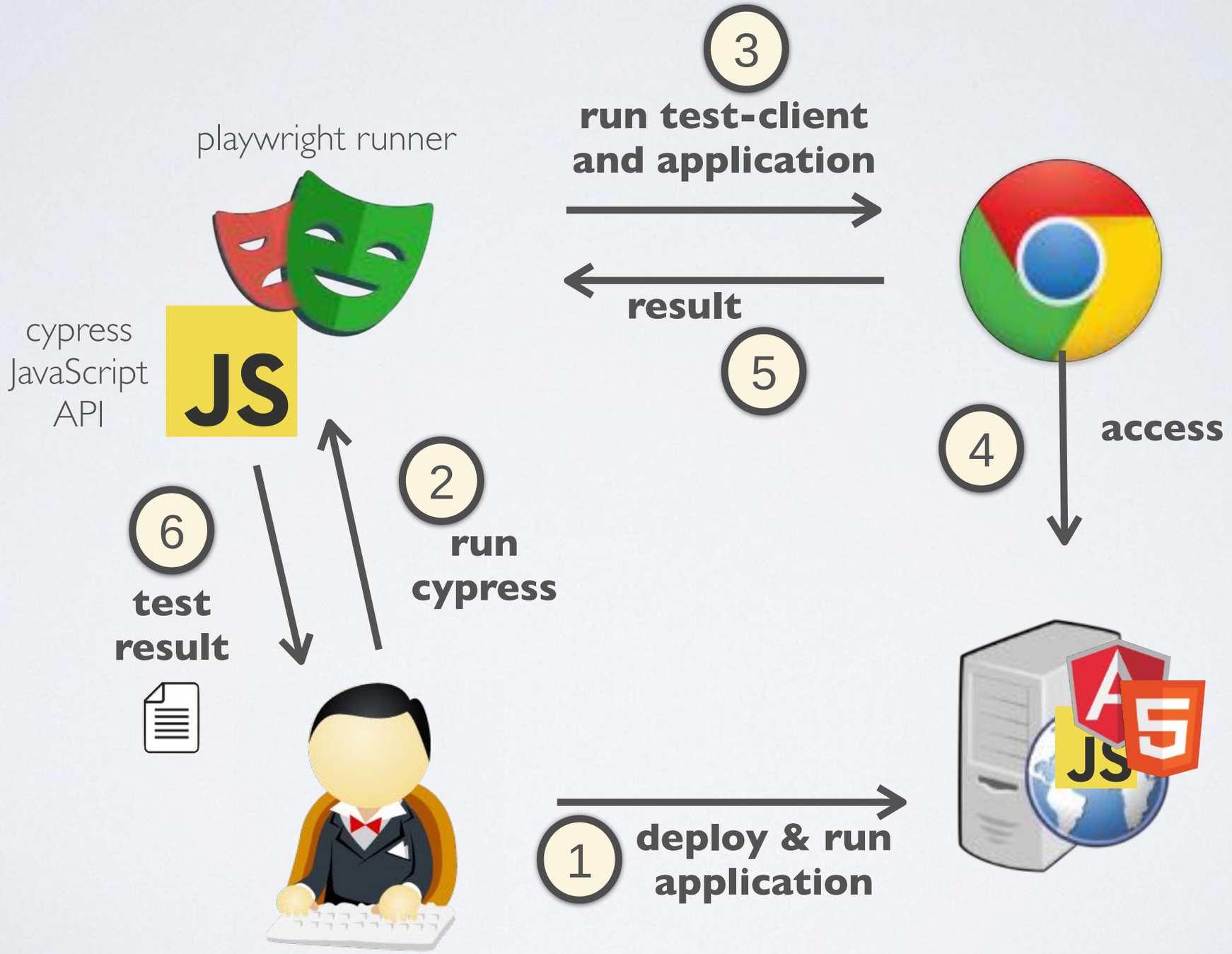


Playwright

https://playwright.dev/

An end-to-end test framework for modern web apps.

Playwright


playwright runner
cypress
JavaScript
API
JS
3
run test-client
and application
result
5
4
access
2
run
cypress
6
test
result
1
deploy & run
application
JS

Modern JavaScript Fundamentals



Java	C	PHP	Ruby	as seen by...
				Java fanboys
				C fanboys
				PHP fanboys
				Ruby fanboys




JS

The Bad Parts

Language Constructs

- Object
- Class
- Method
- Constructor
- Packages
- Inheritance

- Object
- Function

...everything else can be implemented (...as a pattern)

(and modern ES2015+ provides some "syntactic sugar")

Agenda: Modern JavaScript

(for Java Developers)

Functions

Closures

Classes

Modules



Functions

Functions

There are 3 ways to create a function:

- function declaration

```
function doIt(foo, bar) {
    return "result";
}
```

- function expression

```
let doIt = function(foo, bar) {
    return "result";
};
```

- named function expression

```
let doIt = function myFunc(foo, bar) {
    return "result";
};
```

not commonly used today!

Calling the function:

```
console.log(doIt());
```

Function declaration vs. function expression:

- All function declarations are *hoisted* to the begin of the scope and can be used before they are declared.
- Function expressions are not hoisted. Also named function expressions are not hoisted.

Named function expressions:

- Can improve debugging /tracing experience.
- The function name is only visible inside the function.

Arrow Functions: ()=>{...}

ES2015 introduced a compact syntax for function expressions.

ES5 JS

```
var square = function(x){
   return x * x;
}

var add = function(a, b){
   console.log('Adding ...');
   return a + b;
}

var pi = function(){
   return 3.1415;
}

var obj = function(){
   return {props: '!!'};
}
```

2015 ES

```
const square = x => x * x;
const add = (a, b) => a + b;
const pi = () => 3.1415;

const add2 = (a, b) => {
   console.log('Adding ...');
   return a + b;
}

const obj = () => ({prop: '!!'});
```

Differences between regular functions and arrow functions:

- **`this`** is used from the *lexical scope*, they do not have their own binding to **`this`**
- **`call`** and **`apply`** do not bind **`this`**
- have no **`arguments`** and no **`super`**
- calling with **`new`** is not allowed
- no **`prototype`** property

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions
https://javascript.info/arrow-functions

Function Arguments

Functions can declare named arguments:

```
function sayHello(name, date) {…}
```

The caller has to pass these arguments in the the specified order:

```
const name1 = "John";
const date1 = new Date();
sayHello(name1, date1);
```

You can pass fewer arguments as declared.
Remaining arguments are undefined.

```
sayHello(name1);
```

You can pass more arguments as declared. All the arguments are accessible via the `arguments` object in the function.

Function Arguments

"options object pattern"

It is a common pattern to pass several arguments as a single object to functions.

Example: fetch API:

```
fetch(resource)
```

```
fetch(resource, options)
```

```
fetch('https://jsonplaceholder.typicode.com/todos');
```

```
const data = await fetch('https://jsonplaceholder.typicode.com/todos', {
   method: 'POST',
   body: JSON.stringify(...),
   headers: {'Content-type': 'application/json'}
});
```

https://developer.mozilla.org/en-US/docs/Web/API/Window/fetch
https://developer.mozilla.org/en-US/docs/Web/API/RequestInit

Functions are Objects

Functions are regular objects with the additional capability of being callable.

```
function foo(){};
typeof foo; // -> 'function'
foo instanceof Object; // -> true
```

Functions can have properties:

```
function foo(){};
foo.myProp = 'Hello World';
console.log(foo.myProp); //-> 'Hello World'
```

(Functions have a prototype … more about that later)

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Data_structures

Higher Order Functions

- Higher order functions operate on functions
  - functions as parameters
  - functions as return values
- Callback Functions
- Partial Application

```
$("#myBtn").on('click', function() {
  alert("Button Clicked!");
});
```

```
function arrayForEach(array, func) {
  for (let i = 0; i < array.length; i++) {
    func(array[i]);
  }
};
arrayForEach([1,2,3,4,5],
     function(msg){console.log(msg)});
```

```
function add(first, second) {
    return first + second;
}

function defineFirstArg(first, func){
    return function(second){
        return func(first, second);
    }
}

const addOne = defineFirstArg(1, add);
addOne(22); // -> 23
```

Higher Order Functions

More Examples

Wrapping another function:

```
function add(x,y){
  return x + y;
}

function wrapWithLog(fun){
  return function(...args) {
    console.log('Arguments:', args);
    const returnValue = fun(...args);
    console.log('Return Value', returnValue);
    return returnValue;
  }
}

const wrappedAdd = wrapWithLog(add);

console.log(wrappedAdd(41, 1));
```

Composing other functions:

```
function add2(x){ return x + 2; }
function multiplyBy4(x){ return x * 4; }
function divideBy5(x){ return x / 5; }

function pipe(...funs) {
  return function(x){
    const value = x;
    for (const fun of funs){
      value = fun(value);
    }
    return value;
  }
}

cosnt calc = pipe(add2, multiplyBy4, divideBy5);
console.log(calc(3));
```

Note: these examples use the *rest paramater* syntax (`...args`) of ES2015.

`call()` and `apply()`

Functions are objects that can be (re)used in different contexts.

When a function is executed it is bound to a context. The context is represented by the `this` keyword in the function body.

Every function has two properties: `call` and `apply`. These are functions which execute a function with a given context:

```
const context = {prop: 'Hello'};
const doIt = function(arg1, arg2){...}
doIt.call(context, "foo", "bar");
doIt.apply(context, ["foo", "bar"]);
```

Closures



Inner Functions

- Function can be nested inside functions
- Nested functions can access variables in their parent function's scope

```
function outerFunction() {
    const a = 1;
    function innerFunction() {
        return a + 1;
    }
    return innerFunction();
}
const x = outerFunction();
console.log(x);
```

calling the inner function while the outer function is still processing => no closure involved

```
function outerFunction() {
    const a = 1;
    function innerFunction() {
        return a + 1;
    }
    return innerFunction;
}
const x = outerFunction();
console.log(x);
console.log(x());
```

the inner function "closes over" the scope of the outer function

the scope of the outer function is available, even though the outer function has completed

Closures

- Inner functions contain the scope of parent functions even if the parent function has returned.

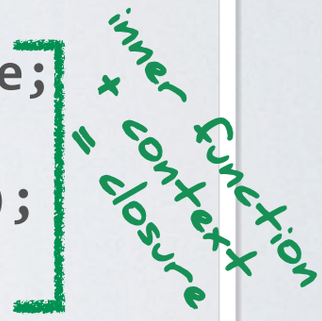
```
let name = 'Bob';
function sayHello() {
     alert(name);
}

name = 'Tim';
sayHello(); // --> Tim
```

- A closure is a special kind of object that combines two things: a function, and the environment in which that function was created. The environment consists of any local variables that were in-scope at the time that the closure was created.

```
var name = 'Bob';
function getGreetingClosure() {
    const text = 'Hello ' + name;
    return function() {
                console.log(text);
            };
}
const sayHello = getGreetingClosure();

name = 'Tim';
sayHello(); // --> Hello Bob
```





Axel Rauschmayer
@rauschma

Following

A closure is a function that doesn't lose the connection to the variables that existed at its birth place.

12:01 PM - 22 Jun 2018

142 Retweets **562** Likes



25 142 562

https://twitter.com/rauschma/status/1010160848491425792

We can think of closures as of "ghosts" or "memories" of the past function calls.

- Dan Abramov, whatthefuck.is/closure

Object Orientation

Creating Objects

Object Literal:

```
const obj = {}
```

Constructor Function:

```
const obj = new Object()
```

not typically used

ES2015: Class Instance:

```
class Person {}

const obj = new Person()
```

Adding Behaviour to Objects

Functions are "first class values", as a consequence we can assign functions to object properties.

```
var firstName = 'Jonas';
var lastName = 'Bandi';
var greet = function(){
        console.log('Hello!');}

var o1 = {
    firstName: firstName,
    lastName: lastName,
    greet: greet
};
```

ES5 JS

2015 ES

```
const firstName = 'Jonas';
const lastName = 'Bandi';
const greet = function(){
        console.log('Hello!');}
const propName = 'myprop';

var o1 = {
    firstName,
    lastName,
    greet,
    say(){console.log('Hi!')};
    [propName]: 'gugus'
};
```

Classes

Classes were added as a language feature in ES2015.

```
class Person {
    constructor(firstName, lastName){
        this.firstName = firstName;
        this.lastName = lastName;
    }

    fullName() {
        return this.firstName + ' ' + this.lastName
    };
}

const pers = new Person('John', 'Doe');
```




```
class Employee extends Person {
    constructor(firstName, company){
        super(firstName, lastName);
        this.company = company;
    }
}

const empl = new Employee('John', 'Doe', 'EvilCorp');
```

Note:

- classes can have static methods
- fields do not have to be declared - but since ES2022 (and with TypeScript) they can be declared
- typically everything is public - but ES2022 introduced private fields (and TypeScript also has "compile-time" private fields)

Classes & Inheritance

```
class Person {

    firstName = '';
    lastName = '';

    constructor(firstName, lastName){
        this.firstName = firstName;
        this.lastName = lastName;
    }

    getfullName() {
        return this.firstName + ' ' + this.lastName
    }

    get fullName() {
      return this.firstName + ' ' + this.lastName;
    }

    set fullName(value) {
      const [firstName, ...rest] = value.trim().split(/\s+/);
      this.firstName = firstName ?? '';
      this.lastName = rest.join(' ');
    }
}

const pers = new Person('John', 'Doe');
console.log(p.fullName);
p.fullName = 'Ada Lovelace';
```

method typically not used both accessor

Note:

- classes can have static methods
- fields do not have to be declared - but since ES2022 (and with TypeScript) they can be declared
- typically everything is public - but ES2022 introduced private fields (and TypeScript also has "compile-time" private fields)
- accessors are not serialized (i.e. `JSON.stringify(pers)`) -> they live on the prototype

```
class Employee extends Person {
    company = '';
    constructor(firstName, lastName, company){
        super(firstName, lastName);
        this.company = company;
    }
}

const empl = new Employee('John', 'Doe', 'EvilCorp');
```

ES2022: Private Fields & Methods

```
class Car {
  #milesDriven = 0
  drive(distance) {
    this.#milesDriven += distance
  }
  getMilesDriven() {
    return this.#milesDriven
  }
}

const tesla = new Car()
tesla.drive(10)
tesla.getMilesDriven() // 10
tesla.#milesDriven // Invalid
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_class_fields

Private fields and methods are part of ECMAScript 2022.
Implemented in all modern browsers and in TypeScript and other transpilers.

Note: classes with private fields cannot be proxied!
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy#no_private_property_forwarding
https://lea.verou.me/2023/04/private-fields-considered-harmful/

The `this` Keyword

For regular functions, the `this` keyword refers to the function's context.
The value of this is determined by how the function is called:

1. If the function is called as a `constructor` (either a class constructor or a function called with `new`) => `this` is a new object
2. If the method is called via `call()` or `apply()` => this is explicitly passed
3. If the function is called as a *method* of an object => this points to that object
4. If none of the above => undefined in *strict mode (typical in modern JS).*
   Legacy JS: `this` is the global object!

Especially in callbacks, it can be tricky to determine this value just by reading the code.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this
http://2ality.com/2017/12/alternate-this.html

Arrow Functions

Arrow function do not explicitly bind **`this`** (aka. *lexical* binding for **`this`**)**:**
The value of **`this`** inside the function body is resolved along the scope chain.
This means that the **`this`** value of the enclosing execution context is used.

```
class Controller {
  constructor() {
    this.count = 0;
  }
  increment() {
    this.count++;
    console.log(this.count);
  }
}

let controller = new Controller();
setTimeout(controller.increment, 0); //-> NaN
```

class method

passed as function

2015 ES

```
class Controller {
  constructor() {
    this.count = 0;
    this.increment = () => {
      this.count++;
      console.log(this.count);
    };
  }
}

let controller = new Controller();
setTimeout(controller.increment, 0); -> 1
```

class field

arrow function

passed as arrow function

Note: functions assigned to *class fields* are not on the prototype and cant be accessed via the **`super`** keyword from a base class!

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions
Real World Use Case: https://reactjs.org/docs/handling-events.html

Classes behind the Scene: Prototypes

```
var o = {};
typeof o.__proto__;
```

Each object has a prototype.

The prototype is an object itself and it is accessible via `__proto__`

The prototype of an object is "hidden" and should not be manipulated.

Properties of objects are resolved by

- looking on the object instance
- looking on the prototype

```
function foo(){};
typeof foo.prototype;
typeof foo.__proto__;
```

Functions have a `prototype` property.

Functions are objects, therefore they also get a `__proto__`.

Prototypal Inheritance

Objects can inherit from other objects

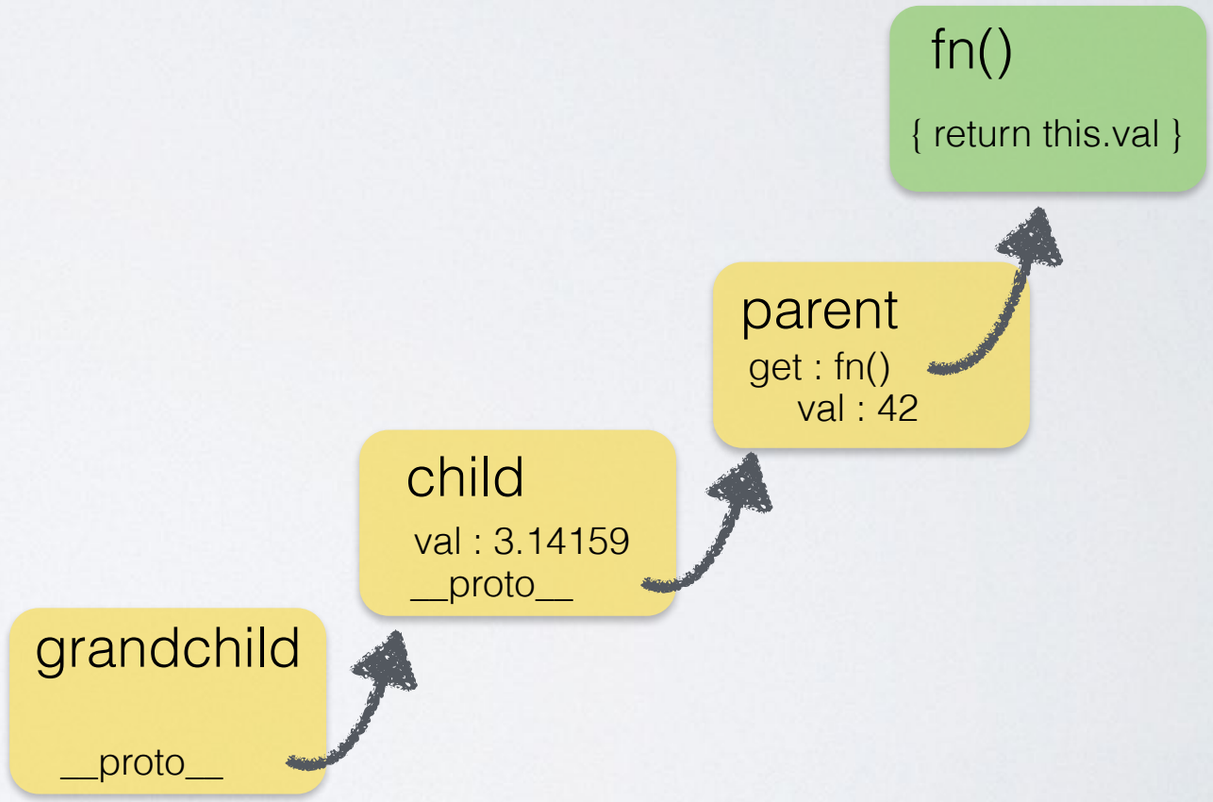
```
var child = Object.create(parent);
```

```
var parent = {
    get: function fn() {
        return this.val;
    },
    val: 42
};

var child = Object.create(parent);
child.val = 3.14159;

var grandchild = Object.create(child);

parent.get(); //42
child.get(); //3.14159
grandchild.get(); //3.14159
```



https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/create

Classes & Prototypes

Classes define a standard way to create object instances that share functionality via prototypes.

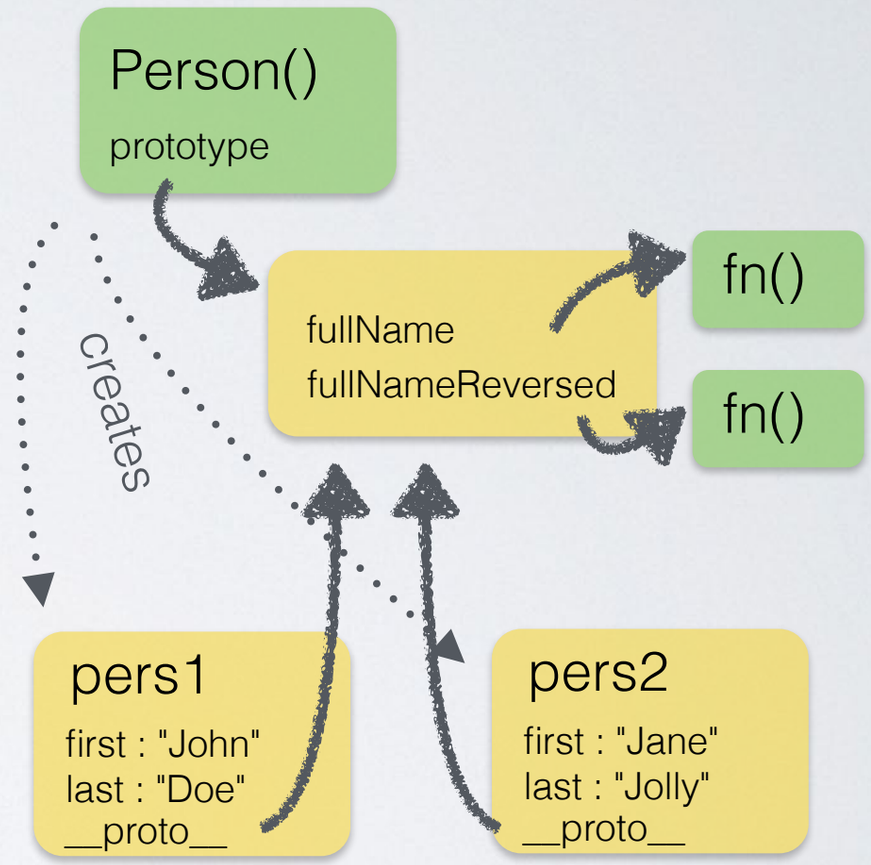
```
class Person {
    constructor(firstName, lastName){
        this.firstName = firstName;
        this.lastName = lastName;
    }

    fullName() {
        return this.firstName + ' ' + this.lastName
    };

    fullNameReversed() {
        return this.lastName + ', ' + this.firstName;
    };

}

const pers1 = new Person('John', 'Doe');
const pers2 = new Person("Jane", "Jolly");
console.log(pers1.fullName());
console.log(typeof Person) // -> function
```



At *runtime* we just have objects and functions attached to their prototypes .
There is no *type* for the class at runtime:

https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Classes



EcmaScript Modules

Classes & Packages in Java

Split code into several files

Isolate units of code & expose APIs

Declare dependencies



resolving during development

```
import javax.persistence.EntityManager;
import com.myorg.domain.MyEntity;
...
EntityManager em = ...
MyEntity myEntity = new MyEntity();
myEntity.setName("Test");
...
em.persist(myEntity);
```



code & libraries

build



resolving at runtime

deployment or platform must provide the dependencies

Dependencies are resolved by the class path / project configuration (pom.xml, gradle file, ...)

EcmaScript Modules (ESM)

In ES5 sharing constructs between files was only possible via the global namespace. Module- and Namespace Patterns helped to restrict what is exposed. Dependencies were managed implicitly.

`library.js`

```
window.doSomething
            = function(){…};
```

`program.js`

```
window.doSomething();
```

ES2015 introduces modules to declare dependencies explicitly.

```
export function doSomething(){
  …
};
```

```
import {doSomething}
        from './library.js';

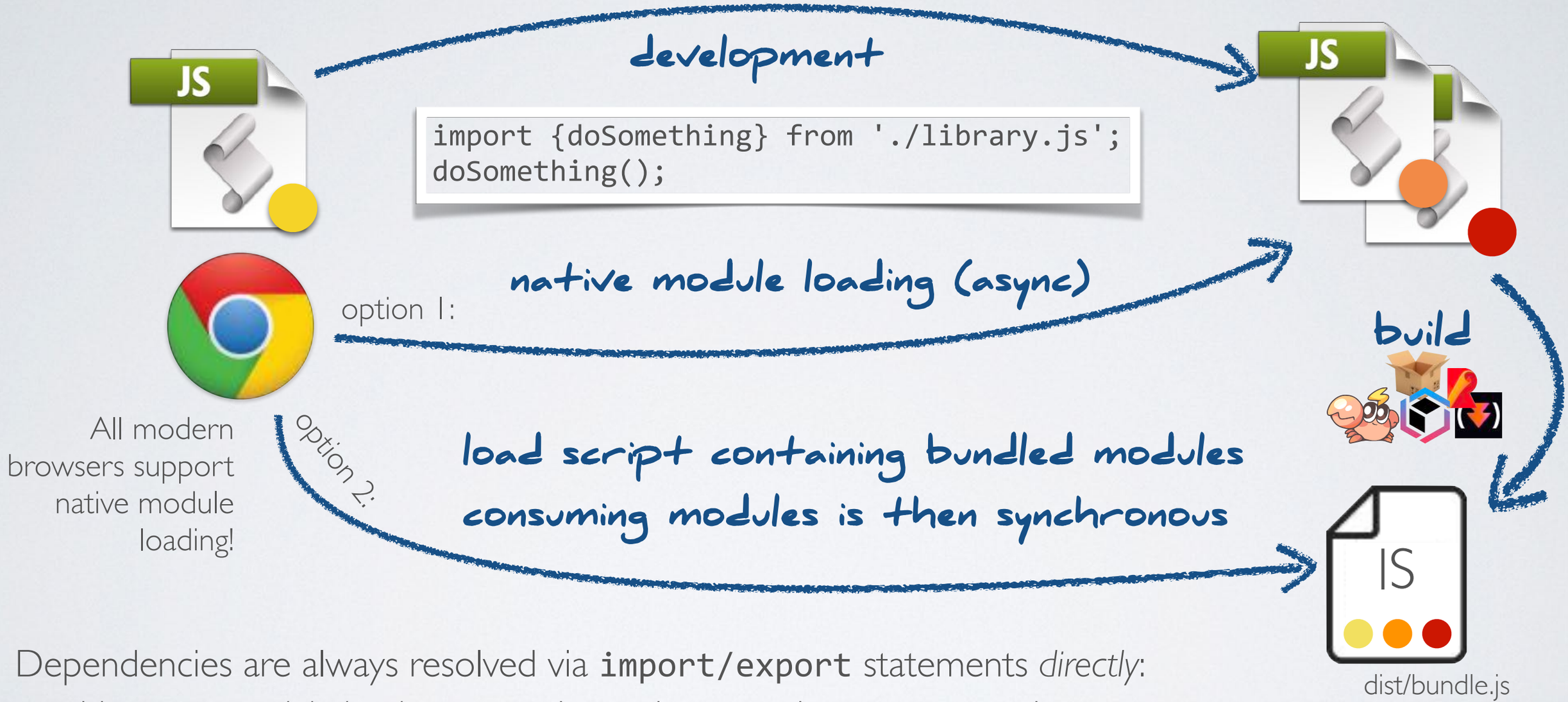
doSomething();
```

At development time each module is a separate file.
At build time modules can (optinally) be bundled with a module bundler (webpack, rollup ...).
At runtime modules can be loaded as separate file (no bundling) or loaded in one or several bundles.

https://hacks.mozilla.org/2018/03/es-modules-a-cartoon-deep-dive/

ES2015 Modules

Special challenge: in the browser scripts are loaded asynchronously!



Dependencies are always resolved via **`import/export`** statements *directly*:

- with *native module loading* at runtime: a import triggers a network request
- with *bundling* at build time: a import defines that the module is included in the bundle (dependency resolution via bundler, typically using node resolution (i.e 3rd party modules are in node_modules installed via npm & package.json)

https://caniuse.com/#search=module

ES2015 Modules Semantics

- Modules are executed when imported
- A modules is only executed once (even when imported several times)
- Modules are always executed in strict mode
- Modules have a top-level scope that is not the global scope
- Modules can `export` bindings to variables, functions or classes
- Modules can `import` bindings from other modules
- `import` bindings are readonly
- `import` statements are hoisted and can't be dynamic

Modules: Syntax

There are different ways to export & import

`library.js`

```
export function
       doSomething(){...};

export let dataObj = {...};

export default () => {...};

export {fun1 as fun2, obj};
```

`program.js`

```
import './library1.js';

import {doSomething, dataObj}
       from './library2.js';

import {doSomething as doIt}
       from './library3.js';

import * as lib3
       from './library4.js';

import defaultExport
       from './library5.js';
```

Opinion: Avoid `export default`: https://basarat.gitbook.io/typescript/main-1/defaultisbad

Default Export vs. Named Export

```
export default class Person {}
```

```
import MyPerson from './person'
```

```
export class Person {}
```

```
import {Person} from './person'
```

Technical difference:
A *named export* is a "live" binding while a *export default* is not.
With a "live" binding the client "sees" later changes by the provider.

Some frameworks require default exports for certain constructs:
React.lazy, route definitions in React Router and Next.js, Angular lazy loading ...

In the TypeScript ecosystem named exports are more widely used.

Personally, I prefer named exports.

Argumentation for named export:
https://basarat.gitbook.io/typescript/main-1/defaultisbad

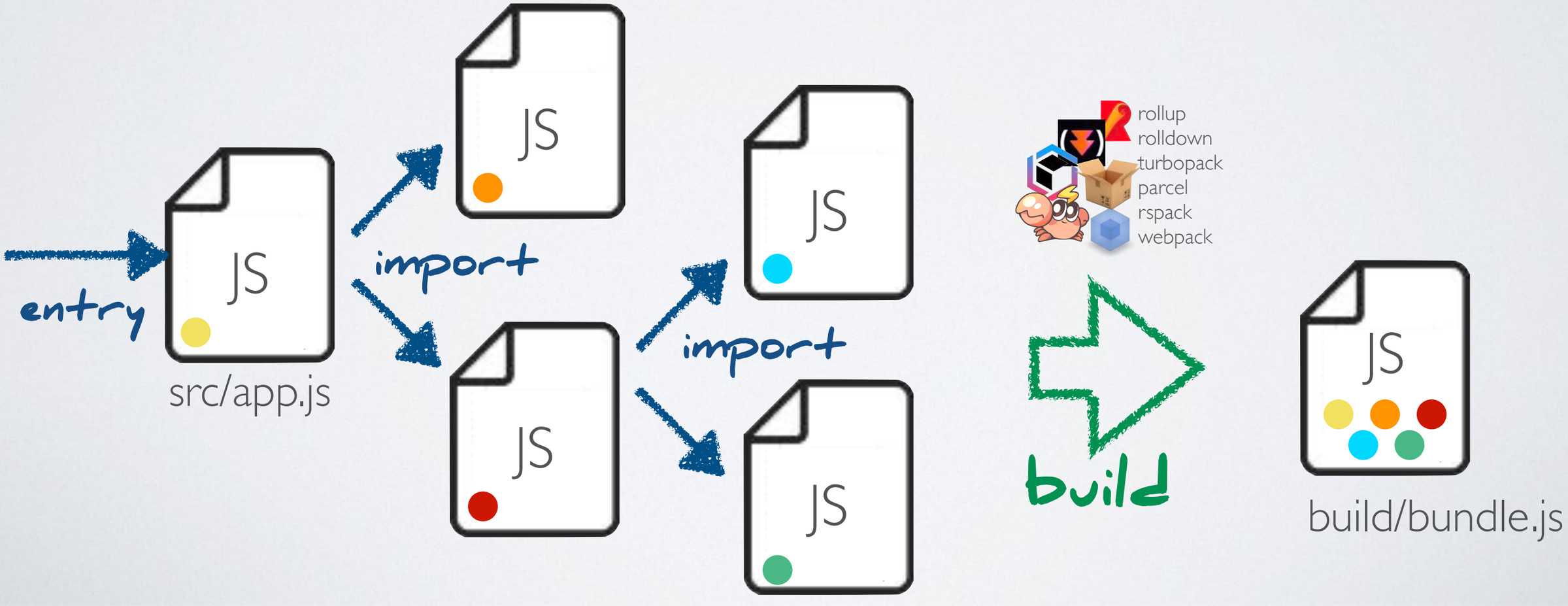
The default export can be error prone (different behavior depending on environment):
https://esbuild.github.io/content-types/#default-interop

Argumentation for default export (however this style guide is not "state of the art"):
https://github.com/airbnb/javascript#modules--prefer-default-export

Modules at Build Time

The primary use-case for modules today is build-tooling.
Modern *bundlers* (Rollup, Parcel, WebPack ...) work with modules:

- build a dependency tree based on fine grained ecma script modules (import/export)
- create coarse grained JavaScript bundles which are optimized to be loaded by a browser.



Modules at Runtime

Module *loading* is not part of the ECMA specification!
It took a while until browser supported modules natively.

Today all modern browsers support *static* module imports:
https://caniuse.com/#feat=es6-module

```
<script type="module" src="app.js"></script>
```

(module scripts are executed after the html parsing has finished, therefore are ideally placed in the <head>
https://html.spec.whatwg.org/multipage/scripting.html#attr-script-defer)

For better performance non-trivial web apps should use a bundler and not native module loading (even with http/2).
https://developers.google.com/web/fundamentals/primers/modules

ECMAScript modules are supported in Node.js v16 and later
https://nodejs.org/docs/latest/api/esm.html

Dynamic Module Loading

Dynamic imports are part of ES2020:

```
import('./second.js')
    .then(m => m.sayHello());
```

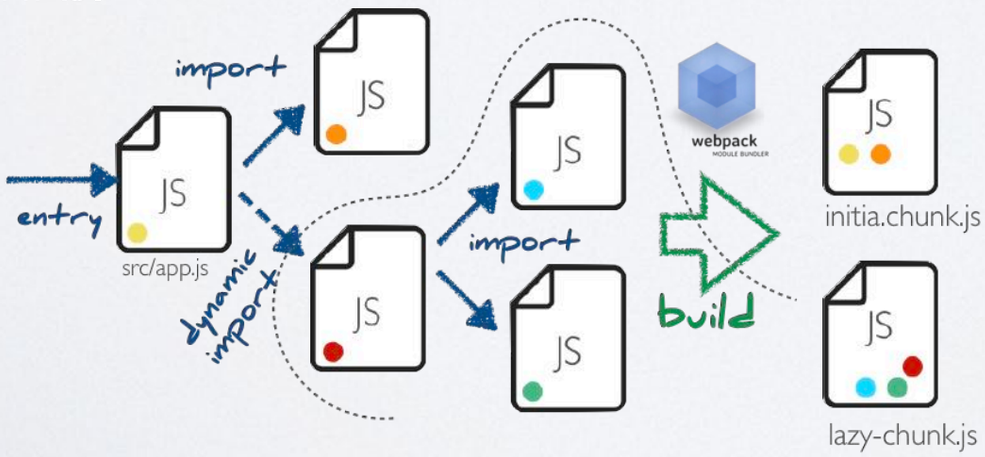
function call

Runtime: supported in all modern browsers.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import#Dynamic_Imports
https://caniuse.com/#feat=es6-module-dynamic-import

Build-Time / Bundling:

Dynamic imports are supported by modern bundlers (webpack, rollup, parcel):
Based on **`import()`** a separate JavaScript bundle is created at build time, that can be asynchronously loaded by the browser when needed.
This is typically used to keep the initial bundle small. This is also called: *code-splitting or lazy-loading* of *chunks*.



https://webpack.js.org/guides/code-splitting/
https://parceljs.org/code_splitting.html
https://rollupjs.org/guide/en#experimental-code-splitting

Import Maps

https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/type/importmap

Html:

```
<script type="importmap">
{
    "imports": {
        "lodash/chunk": "https://cdn.jsdelivr.net/npm/lodash-es@4.17.21/chunk.js"
    }
}
```

JS (directly loaded in the browser, no bundler):

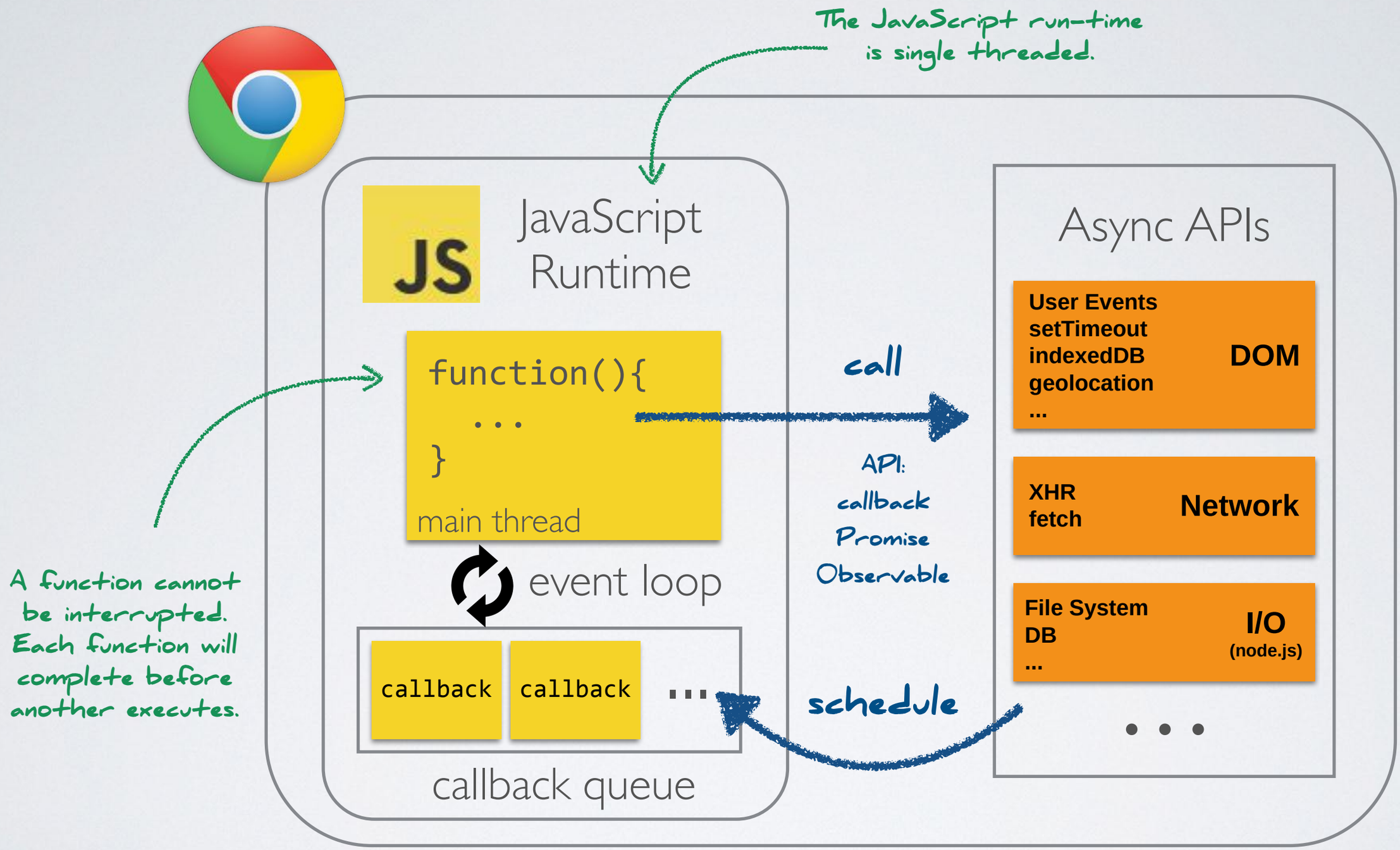
```
import chunk from "lodash/chunk";
console.log("Groups of two:", chunk([1, 2, 3, 4, 5], 2));
```

Enables decoupling of source code from URL/Version.
Makes code portable between Browser and Node/Bundlers.



AJAX & Async JavaScript

Concurrency in JavaScript: Event Loop



https://www.tedinski.com/2018/10/16/concurrency-vs-parallelism.html
What the heck is the event loop anyway? - https://www.youtube.com/watch?v=8aGhZQkoFbQ
Interactive Visualization: http://latentflip.com/loupe/
Jake Archibald: In The Loop: https://www.youtube.com/watch?v=cCOL7MC4Pl0
https://jakearchibald.com/2015/tasks-microtasks-queues-and-schedules/

Note: there is also the differentiation between macro- and micro tasks on the event loop: https://javascript.info/event-loop

AJAX with Callbacks

using jQuery as an example

```
$.get('http://localhost:3001/comments', (data) => {
    console.log(data);
});
```

```
$.post('http://localhost:3001/comments',
      {text: 'test - ' + new Date()},
      () => {
         console.log('POST!');
});
```

`fetch`: a modern AJAX API

`fetch` is a new API available in modern browsers that is more elegant than `XMLHttpRequest`.

```
fetch('https://swapi.info/api/people/1')
    .then(response => response.json())
    .then(data => console.log('SUCCESS', data))
    .catch(error => console.log('ERROR:', error));
```

should first check if response is ok!

`fetch` is available in all modern browsers.

`fetch` is built on Promises.

alternative API using `Request`:

```
const request = new Request('https://swapi.info/api/people/1');
fetch(request)
    .then(response => response.json())
    .then(data => console.log('SUCCESS', data))
```

Browser support:
http://caniuse.com/#search=fetch

https://developer.mozilla.org/en-US/docs/Web/API/Window/fetch
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Promise-based Backend Access: `fetch` vs. `axios` vs. `ky`

`fetch` is a modern built-in browser API:

```
const res = await fetch('https://swapi.co/api/people/1/');
if (!res.ok) {
  throw new Error('Network response was not ok');
} else {
  const data = await res.json();
  console.log('SUCCESS', data);
}
```

non-2XX status is NOT an error!

`fetch` is available in all modern browsers (http://caniuse.com/#search=fetch).

For many projects, it makes sense to use a http library with more features:

- `ky`: https://github.com/sindresorhus/ky (modern, based on fetch)
- `axios`: https://github.com/axios/axios (legacy, based on XHR)

```
import ky from 'ky';
const response = await ky.get('http://www.example-api.com');
const data = await response.json<ResponseType>();
```

Features of http libraries: automatic json data transformation, status handling, http interceptors, response timeout, easily cancellable ...

Promises



- A promise represents the result of an asynchronous operation.

Promises

- Promises are a way how to deal with asynchronity http://andyshora.com/promises-angularjs-explained-as-cartoon.html

One morning, a father says to his son: "Go and get the weather forecast, son!"



Promises

A promise represents the result of an asynchronous operation.

- Counterparts: `Future<>` in Java or `Task<>` in .NET (TPL)

Promises enable a programming model that is an alternative to callbacks.

- Asynchronous functions can return a value without blocking (so they look much more like synchronous functions)

- The return value is a promise that represents the final value that is possibly not yet known

Promises

Promises are a way to model asynchronous operations.
A promise is an object that represents the completion of an asynchronous operation and the resulting value.



Many different libraries implement promises for ES5.
(i.e. bluebird, Q, AngularJS, jQuery v3, ...)

https://promisesaplus.com/implementations



`Promise` is a built in object in ES2015.

```
const p = new Promise((resolve, reject) => {
   setTimeout(() => resolve('result), 1000);
});
p.then((value) => console.log(value));
p.catch((error) => console.log('Oh, crap!));
```

There are several polyfills for ES2015 promises.
i.e: https://github.com/taylorhakes/promise-polyfill

Using Promises

Consuming a promise:

```
promiseMeSomething()
    .then((value) => {
       // success handler
    })
    .catch((error) => {
      // error handler
    });
```

Provide a promise (EcmaScript 2015):

```
function promiseMeSomething(){
  const promise = new Promise((resolve, reject) => {
     // resolve the promise later
      setTimeout(function () { resolve("Success!");}, 1000);
  });
  return promise;
}
```

Evolution of ECMAScript Promises

ECMAScript 2015:

Construction:
- `new Promise((resolve, reject) => {})`
- `Promise.resolve(), Promise.reject()`

Handling/Flow:
- `Promise.prototype.then()`
- `Promise.prototype.catch()`

Combinators:
- `Promise.all()`
- `Promise.race()`

DOM API: Abort Controller (2017) -> can be used to cancel a Promise API

ECMAScript 2018: `Promise.prototype.finally()`

ECMAScript 2020: `Promise.allSettled()`

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/allSettled

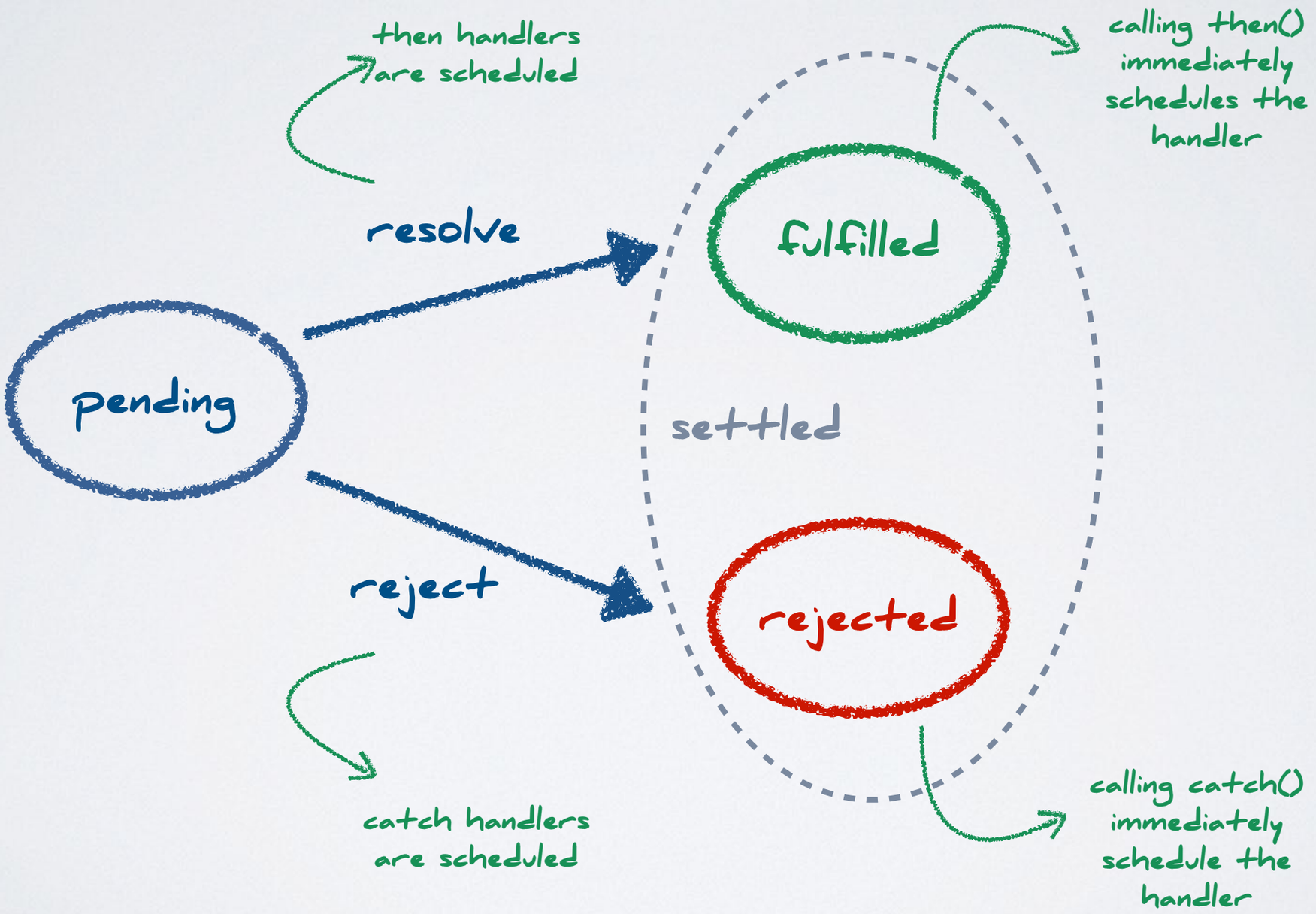
ECMAScript 2021: `Promise.any()`

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/any

ECMAScript 2024: `Promise.withResolvers()`

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/withResolvers

Promises represent a Statemachine



Chaining Async Operations

Callback-Hell:

(aka Pyramid of Doom)

```
api((result) => {
    api2((result2) => {
        api3((result3) => {
            // do work
        });
    });
});
```

Promise Chaining:

If a "then-callback" returns another promise, the next "then" waits until it is settled.

```
api()
  .then((result) => {
    return api2();
  })
  .then((result2) => {
    return api3();
  })
  .then((result3) => {
     // do work
  })
  .catch(function(error) {
     //handle any error
});
```

```
api().then(api2)
     .then(api3)
     .then(/* do work */);
```

Combining Multiple Actions

```
const firstData = null;
const secondData = null;

const responseCallback = () => {
 if (!firstData || !secondData) return;
 // do something
}

getData("http://url/first", (data) => {
  firstData = data;
  responseCallback();
});

getData("http://url/second", (data) => {
  secondData = data;
  responseCallback();
});
```

cumbersome orchestration

```
const promise1 = getData("http://url/first");
const promise2 = getData("http://url/second");

Promise.all([promise1, promise2])
    .then((arrayOfResults) => {…})
    .catch((errOfPromise) => {…});

Promise.race([promise1, promise2])
    .then((valOfPromise) => {…})
    .catch((errOfPromise) => {…});

Promise.any([promise1, promise2])
    .then((valOfPromise) => {…})
    .catch((errOfPromise) => {…});

Promise.allSettled([promise1, promise2])
    .then((valOfPromise) => {…})
    .catch((errOfPromise) => {…});
```

`Promise.all`: short-circuits when an input value is rejected

`Promise.race`: short-circuits when an input value is settled

`Promise.any`: short-circuits when an input value is fulfilled

`Promise.allSettled`: waits until all input valies are fulfilled or rejected

Promise Chaining

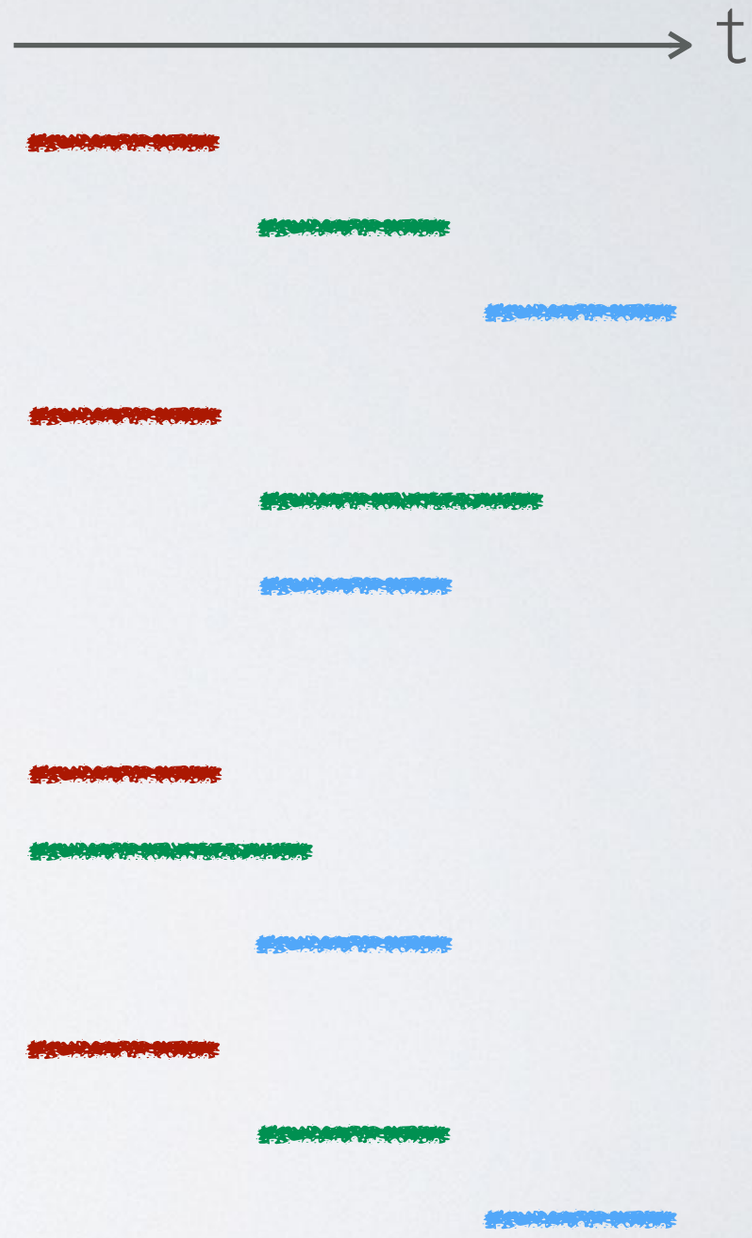
pay attention to the timing!

```
doSomething().then(function () {
    doSomethingElse();
}).then(finalHandler);
```

```
doSomething().then(function () {
  doSomethingElse();
}).then(finalHandler);
```

```
doSomething()
   .then(doSomethingElse())
   .then(finalHandler);
```

```
doSomething()
   .then(doSomethingElse)
   .then(finalHandler);
```



https://pouchdb.com/2015/05/18/we-have-a-problem-with-promises.html

"Cancelling" a Promise API

using **AbortController** and **AbortSignal**

```
const controller = new AbortController();
const signal = controller.signal;
setTimeout(() => controller.abort(), 1000);

function promiseMeSomething(signal){
  if (signal.aborted) {
    return Promise.reject('aborted');
  }

  return new Promise((resolve, reject) => {
    setTimeout(() => resolve("Success!"), 2000);

    signal.addEventListener('abort', () => {
      reject('aborted!')
    });
  });
}

promiseMeSomething(signal)
  .then((value) => console.log(value))
  .catch((error) => console.log(error));
```

https://developer.mozilla.org/en-US/docs/Web/API/AbortSignal

AbortController is a DOM API: https://dom.spec.whatwg.org/#aborting-ongoing-activities

Promise.withResolvers()

(available since ES2024)

resolve and reject is only available inside the Promise!

```
const promise = new Promise((resolve, reject) => {
   // resolve the promise from within
    setTimeout(function () { resolve("Success!");}, 1000);
});
// cannot resolve the promise here ...
```

```
const { promise, resolve, reject } = new Promise.withResolvers();

setTimeout(resolve('A value!'), 3000);
```

resolved from outside!

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/withResolvers



async / await

async / await

```
function sleep(millis){
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve(), millis);
  });
}

async function start(){
  await sleep(1000);
  console.log('finished!');
}

start();
```

Note: `await` can be used inside an `async` function or on top-level

An `async` function (implicitly) returns a promise, which is resolved with the return value.

An `async` function can contain an `await` expression, that pauses the execution of the `async` function and waits for the passed `Promise`'s resolution, and then resumes the `async` function's execution and returns the resolved value.

The behavior of the event loop executing the code remains the same as in a promise chain.

`async` / `await` is a language feature of ES2017.
It is natively supported in all modern browsers.

https://caniuse.com/#feat=async-functions

async / await

`async`/`await` provides convenient language support on top of any Promise based API.

```
try {
   const response = await axios.get(API_URL);
   console.log(response);
} catch (error) {
   console.log(error)
}
```

```
const response = await axios.post(API_URL, payload);
console.log(response);
```

```
const response = await axios.delete(`${API_URL}/${id}`);
console.log(response);
```

Note: **`await`** can only be used inside an **`async`** function

(or on top level of modules, see https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await#top_level_await)

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

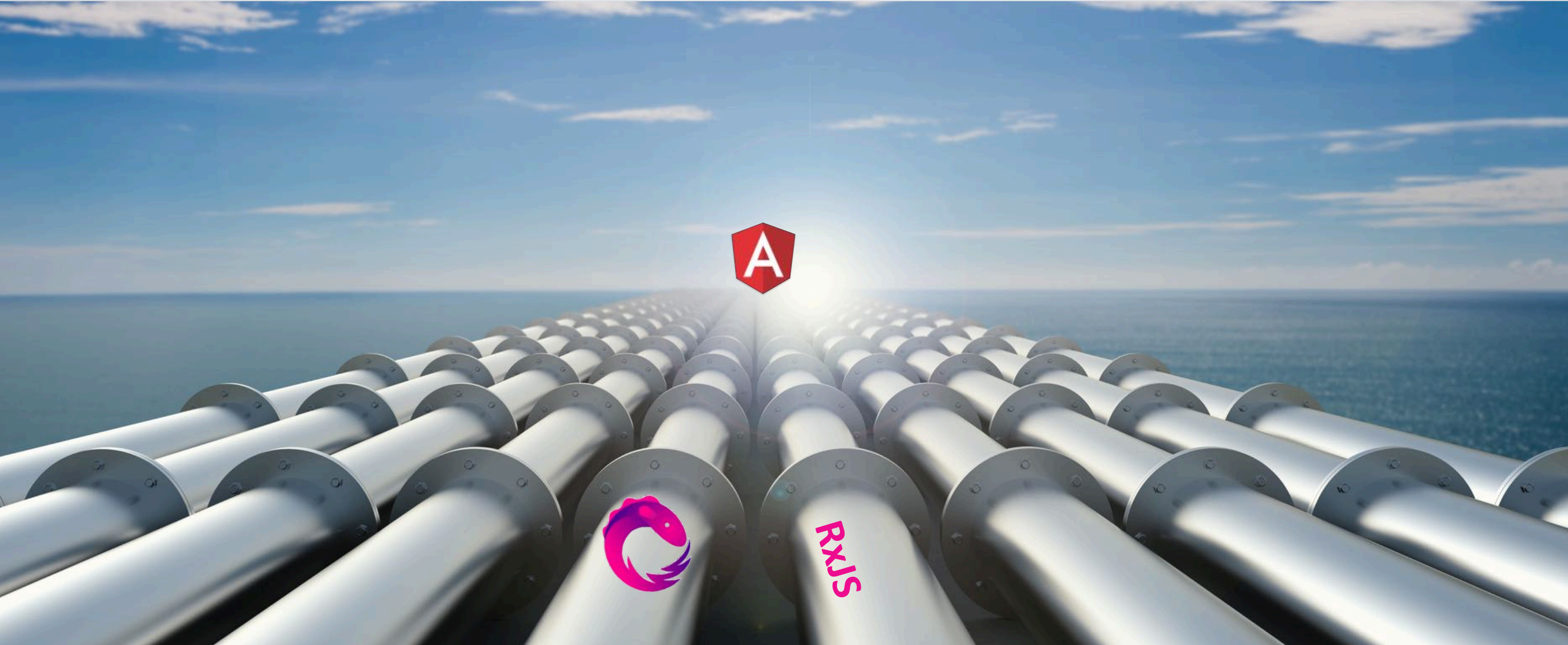
async / await is still promises, but with a really nice syntax!

```
function sleep(millis){
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve(), millis);
  });
}

async function start(){
  await sleep(1000).then(() => console.log('waking up!'));
  console.log('finished!');
}

start();
```

The behavior of the event loop executing the code remains the same as in a promise chain.



RxJS / Observables

Observables

An observable represents a multi-value push protocol:

	Single	Multiple
Pull	`Function`	Iterator
Push	`Promise`	`Observable`

Typical scenario: an asynchronous stream of events.

```
const observable =
   rxjs.fromEvent(
      document.getElementById('my-button'),'click'
   );

observable.subscribe(x => console.log(x));
```

Observables in ECMAScript

RxJS is the "de-facto" implementation of observables today.



RxJS

https://rxjs.dev/ ??? -> https://rxjs-dev.firebaseapp.com

RxJS is a library for composing asynchronous and event-based programs by using observable sequences.

There is a proposal to standardize **`Observable`** as built-in type in a future version of ECMAScript.

There are different competing libraries with similar reactive programming concepts: Beacon.js, xstream, …

https://github.com/tc39/proposal-observable

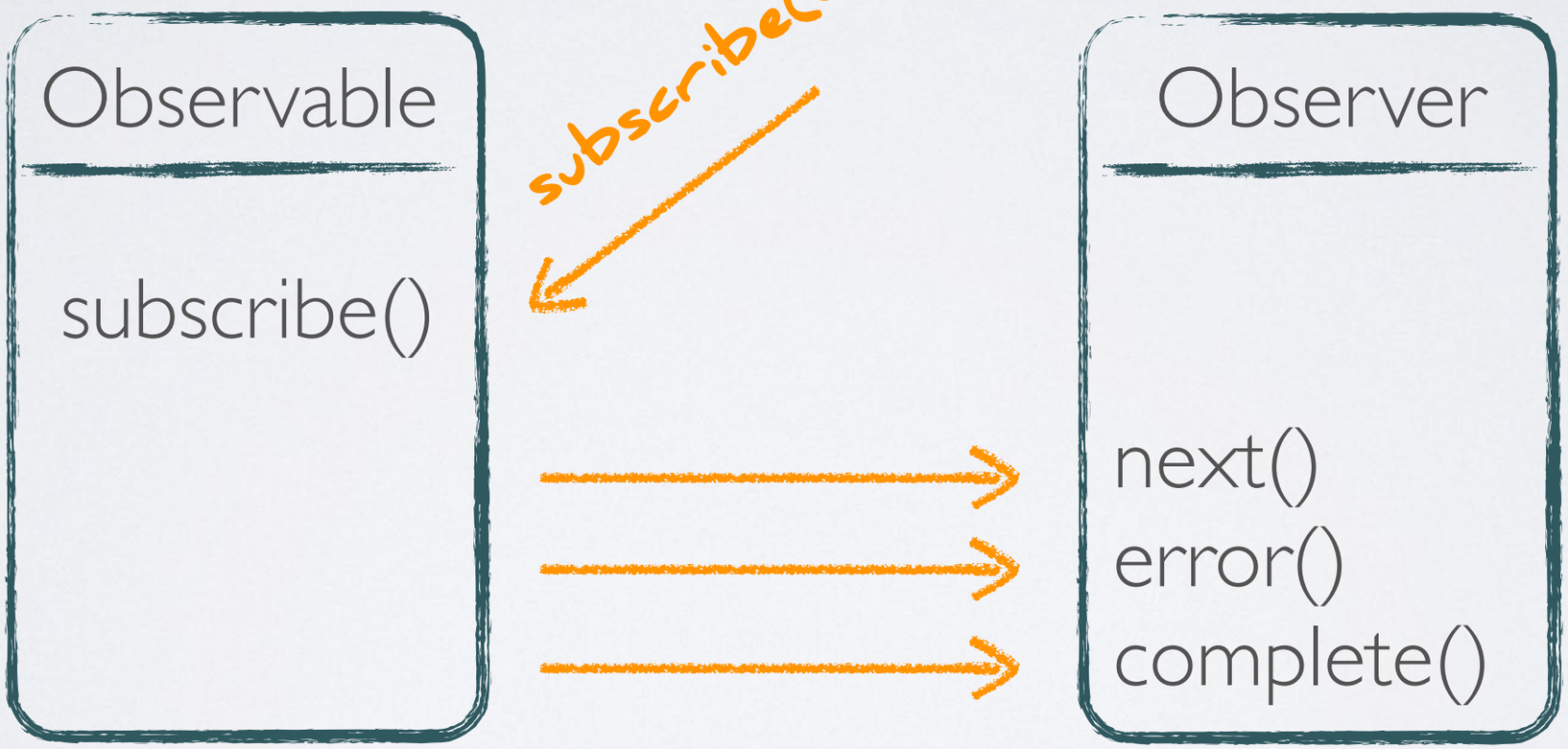
http://reactivex.io/
https://baconjs.github.io/
http://staltz.com/xstream/

Characteristics of Observables

- Represents any number of values over any amount of time
- Observables are able to deliver values either synchronously or asynchronously
- Observables are lazy
- Observables can be cancelled
- Observables can be composed with higher-order combinators

RxJS:
"The Observer pattern done right"

http://reactivex.io/



The power of RxJS are the operators on observables and the composability of observables.

Using Observables

Consume an observable:

```
var value$ = startAsyncOperation();
value$.subscribe(
  value => document.getElementById("content").innerText += value,
  error => console.log('Error: ' + error),
  () => console.log('Completed!')
);
```

Provide an observable (using RxJS):

```
function startAsyncOperation(){
   var value$ = new rxjs.Observable(
     observer => {
       setTimeout(() => observer.next("This is first value!"), 1000);
       setTimeout(() => observer.next("This is second value!"), 2000);
       setTimeout(() => observer.next("This is third value!"), 3000);
       setTimeout(() => observer.complete(), 4000);
     });
   return value$;
}
```

Note: In typical application programming you *never* have to create an observable like this.
You either use factory functions or you consume an API that exposes an "event source" as an observable.

Observables vs. Promises

An observable can be an alternative to a promise:
a stream that pushes exactly one result.

```
const observable = new rxjs.Observable(
   observer => setTimeout(
        () => {
             observer.next(42);
             observer.complete();
        },
        2000);
);

observable.subscribe(x => console.log(x));
```

Angular exposes observables for backend access via `HttpClient` service.

Creating Observable Streams

Creation Functions:

```
of, from, fromEvent,
interval, timer ...
```

Manual Creation:

```
new Observable(observer => {
  observer.next(1);
  observer.next(2);
  observer.complete();
});
```

```
new Observable(observer => {
  setTimeout(() =>  observer.next(1), 1000);
  setTimeout(() =>  observer.next(2), 2000);
  setTimeout(() =>  observer.complete(), 3000);
});
```

Note: In typical application programming you *never* have to manually create an observable.
You either use creation functions or you consume an API that exposes an "event source" in an observable.

Subscription & Unsubscribe

A subscription is modeled by a **`Subscription`** object.

```
const subscription = interval(1000)
  .subscribe(
    value => console.log(value),
    error => console.log(error),
    () => console.log('COMPLETED')
  );

setTimeout(() => {
  console.log(subscription.closed);
  subscription.unsubscribe();
}, 4000)
```

There is a potential memory leak, if you are not unsubscribing from an Observable!

Cold vs. Hot Observables

A cold observable creates the producer:

```
var cold = new Observable((observer) => {
  var producer = new Producer();
  // have observer listen to producer here
});
```

Each subscription creates it's own producer. The producer only starts producing with the subscription.

A hot observable closes over the producer:

```
var producer = new Producer();
var hot = new Observable((observer) => {
  // have observer listen to producer here
});
```

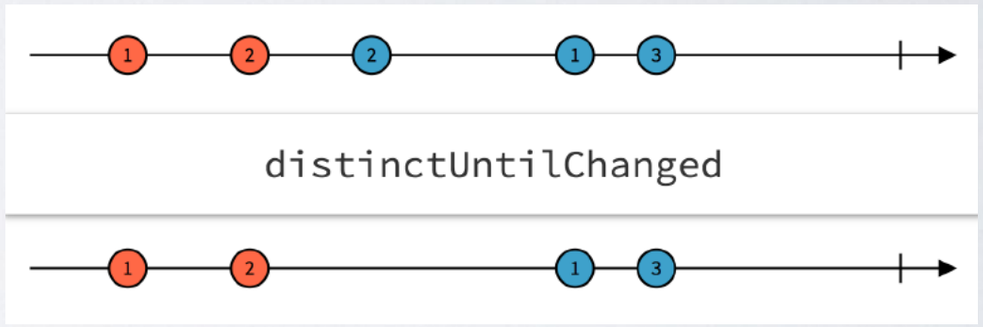
The producer produces values independent of subscriptions.

https://medium.com/@benlesh/hot-vs-cold-observables-f8094ed53339

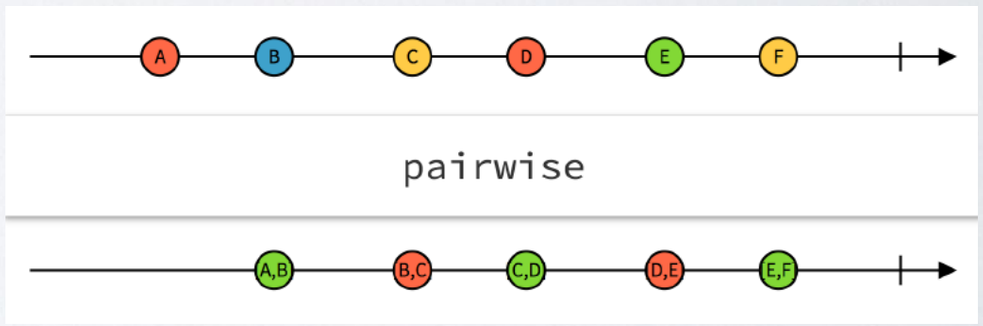
Transformation Operators

```
const {filter, map} = rxjs.operators;
const o1 = rxjs
            .interval(1000)
            .pipe(
                 filter(v => v % 2 === 0),
                 map(v => v * 2)
             );
```

```
const {distinctUntilChanged} = rxjs.operators;
const transformed = o1.
            .pipe(
               distinctUntilChanged()
            );
```




```
const {pairwise} = rxjs.operators;
const transformed = o1
            .pipe(
               pairwise()
            );
```




```
transformed.subscribe(
  v => console.log(v)
);
```

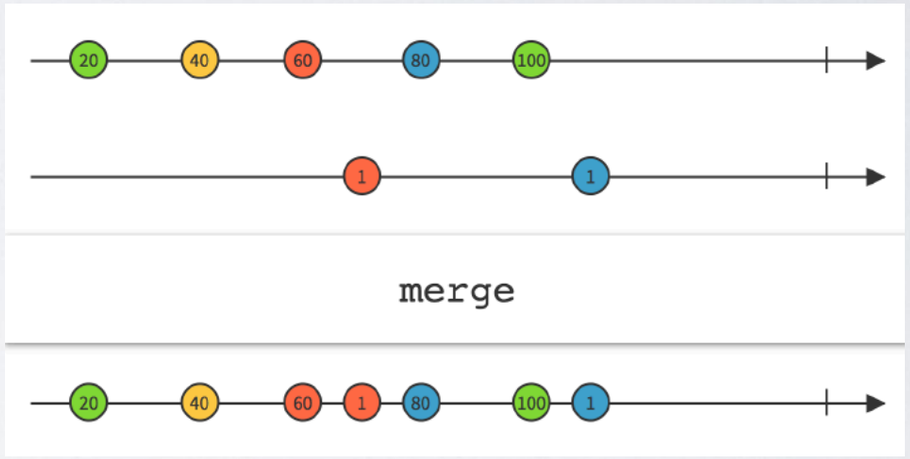
https://rxjs.dev/guide/operators
http://rxmarbles.com/

Combination Operators

```
const o1 = rxjs.interval(1000);
const o2 = rxjs.interval(1000)
  .delay(500)
  .map(v => v * 1000);
```

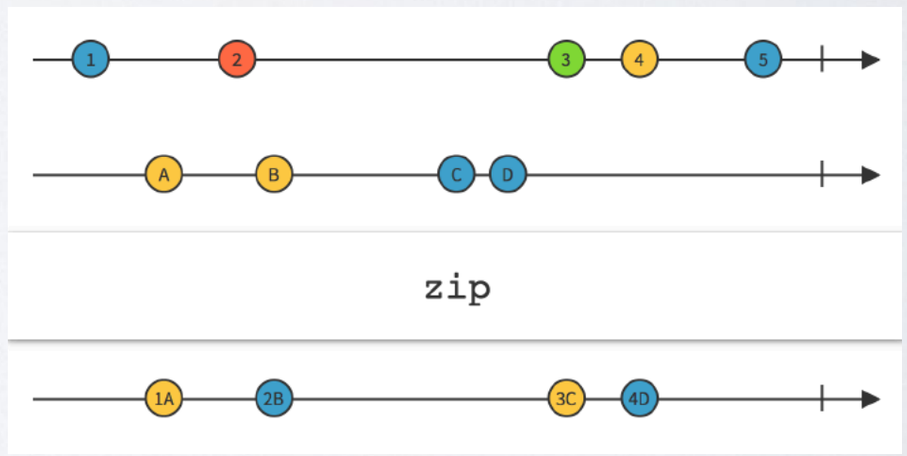
```
const combined = rxjs.merge(o1,o2);

combined.subscribe(
  v => console.log(v)
);
```




```
const combined = rxjs.zip(o1,o2);

combined.subscribe(
  v => console.log(v)
);
```



http://reactivex.io/documentation/operators
http://rxmarbles.com/

Higher Order Observables

"Observables of Observables"

```
function api1(){
  return rxjs.of(42).pipe(delay(2000));
}
function api2(){
  return rxjs.of(43).pipe(delay(1000));
}
function api3(){
  return rxjs.of(44).pipe(delay(500));
}

stream = rxjs.from([api1, api2, api3]);
```

```
stream
  .pipe(mergeMap(x => x()))
  .subscribe(console.log);
```

out:
44, 43, 42

```
stream
  .pipe(concatMap(x => x()))
  .subscribe(console.log);
```

out:
42, 43, 44

```
stream
  .pipe(switchMap(x => x()))
  .subscribe(console.log);
```

out:
44

